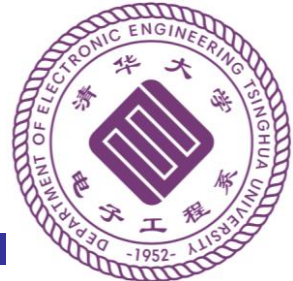




数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第三讲 组合逻辑分析与设计

李学清

xueqingli@tsinghua.edu.cn

助教：曾令安*/唐文骏/陈仲豪

*本节内容负责助教

2024年9月23日

通知、答疑

- 后选课的同学
 - 自学、网络学堂**补交**第一次作业
- **微信实名**

唐文骏, 158 0118 6917, twj21@mails.tsinghua.edu.cn, 电子工程馆7-305

陈仲豪, 183 5975 7529, czh23@mails.tsinghua.edu.cn, 电子工程馆4-301

曾令安, 188 0135 7562, zla23@mails.tsinghua.edu.cn, 电子工程馆7-305

注意：7层西侧与弧形中厅走廊常锁，请通过其他层前往西侧，再走楼梯/电梯到达

习题课/答疑：

1. 周六晚7:00 – 9:00集中答疑、习题课
 - 答疑每周1次（第2周起），系馆9-206
 - 习题课时间地点待定，每主题一次，讲习题中问题，讲解后继续答疑
2. 网络学堂课程讨论区答疑
3. 助教/教师邮件答疑

课程回顾-布尔代数

- 数的编码和表示
 - 二进制编码
 - 负数的表示和计算
 - 有符号数与补码
- 布尔逻辑的计算
 - 布尔表达式
 - 布尔表达式的化简方法
 - 提问：什么是最简表达式？

提出问题

- 你的芯片里需要一个64位的加法器
 - $A=[A_{63}A_{62}\dots A_0]$, $B=[B_{63}B_{62}\dots B_0]$
 - $S=A+B$
- 某个供应商提供了一种加法器，是否足够好？
 - 性能 (速度 or 延迟)
 - 耗费 (功率 or 能量, 面积)
 - 其它 (工艺, 可靠, 供应电压, etc.)
- 你能自己设计吗？

组合逻辑的作用



X $\xrightarrow{f}$

Y

输入
布尔比特

输出
布尔比特

周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	待定	期中考试 (TBD)
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

本节课学习目标

- 理解

- 布尔代数到组合逻辑的映射
- 组合逻辑电路的评价指标和非理想因素

- 掌握

- 组合逻辑电路的分析和设计方法，实现常见组合逻辑电路的应用和优化

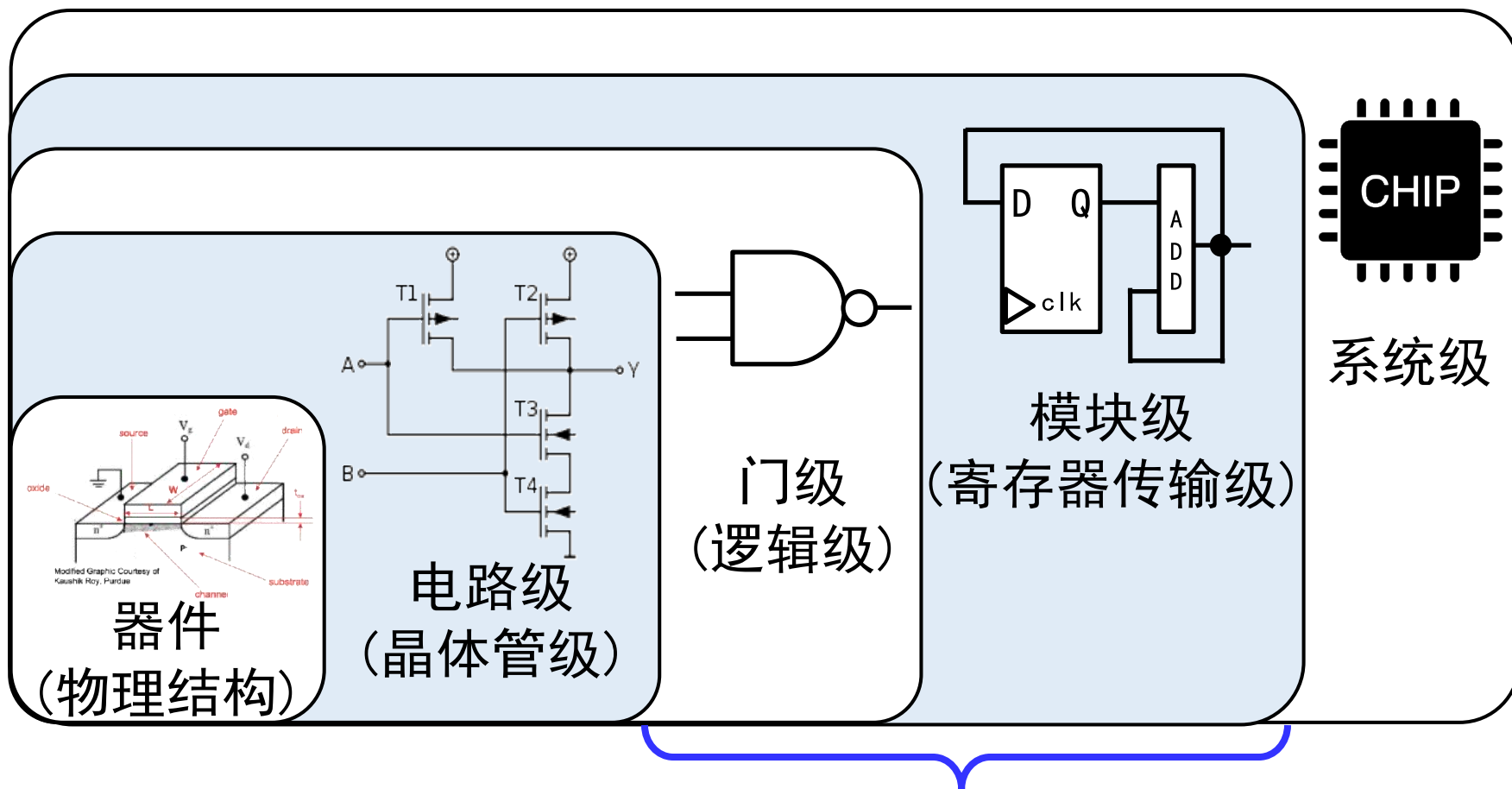
- 树立

- 跨层次设计理念和折衷优化思想

本节课目录

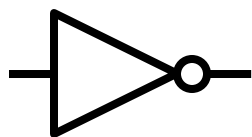
- 从布尔表达式到开关电路
- 定义、分析、设计
- 评价指标和冒险
- 常用组合逻辑模块

布尔表达式与开关电路的实现

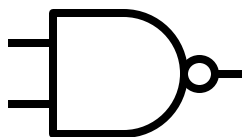


本课程重点关注内容

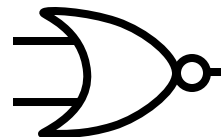
逻辑门-门级基础器件



NOT



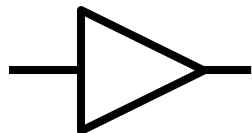
NAND



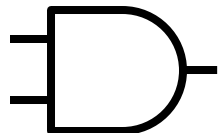
NOR



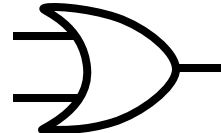
XNOR



Buffer



AND

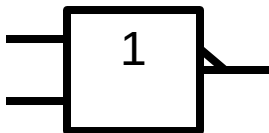


OR

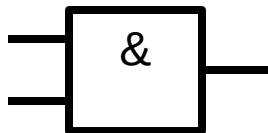


XOR

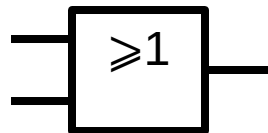
推荐
使用
符号



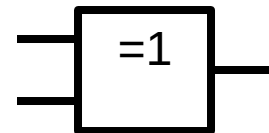
非门



与门

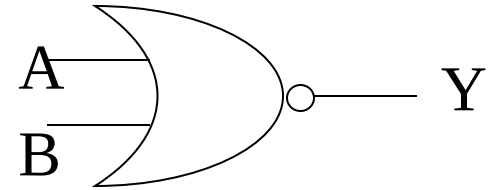
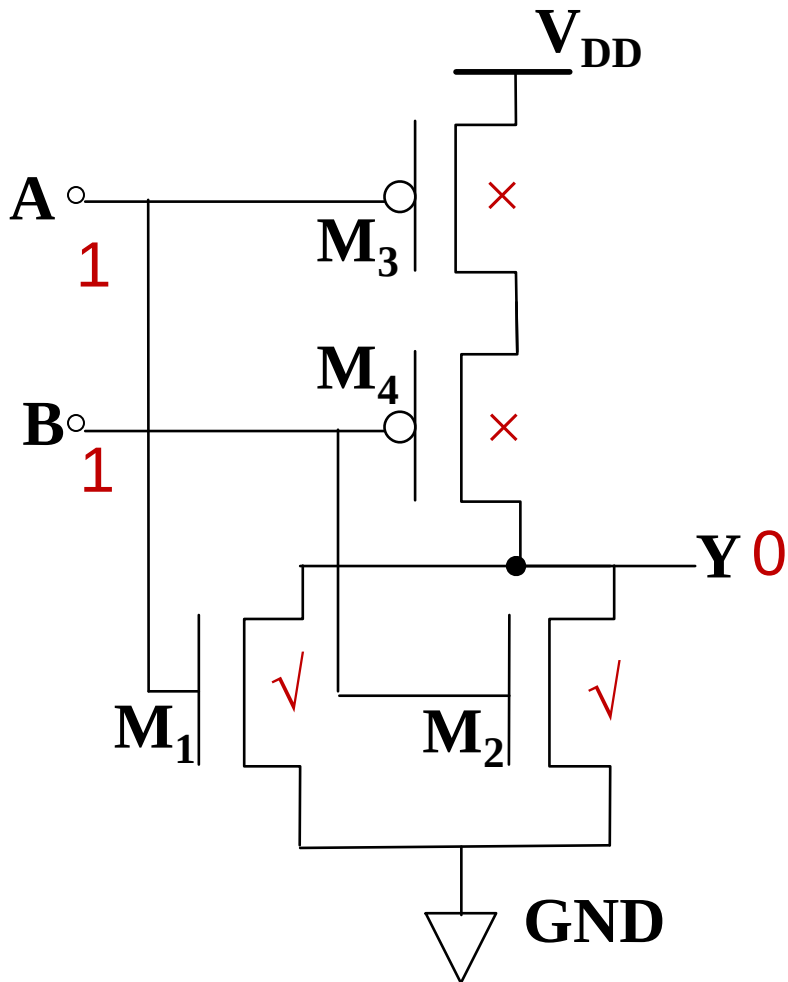


或门



异或门

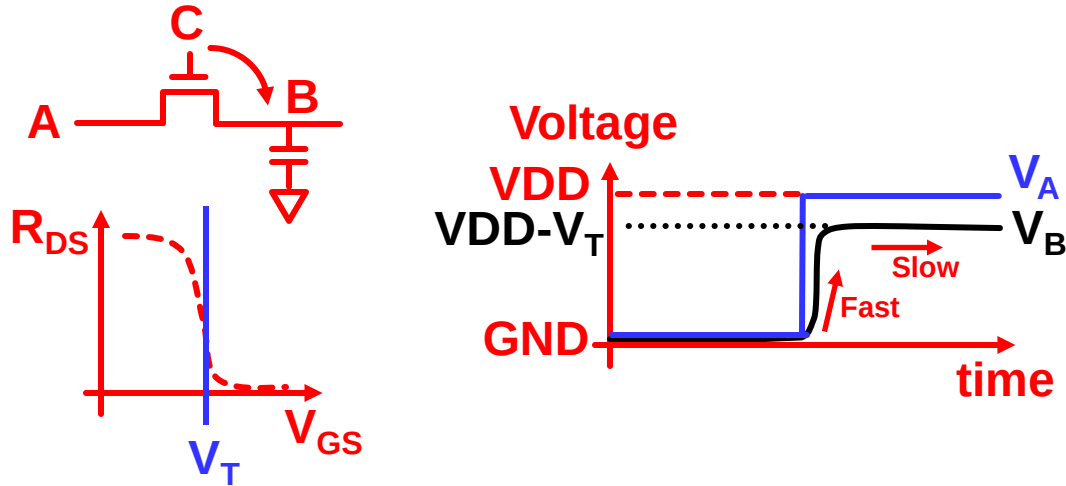
举例：CMOS或非门



<i>A</i>	<i>B</i>	M_1	M_2	M_3	M_4	<i>Y</i>
L	L	Off	Off	On	On	H
L	H	Off	On	On	Off	L
H	L	On	Off	Off	On	L
H	H	On	On	Off	Off	L

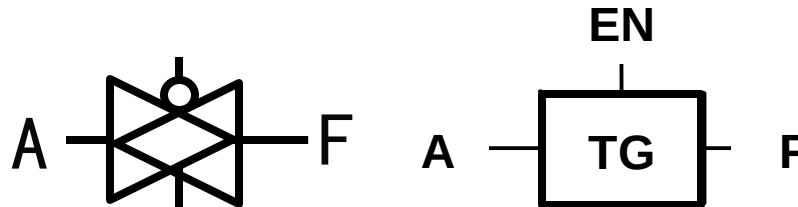
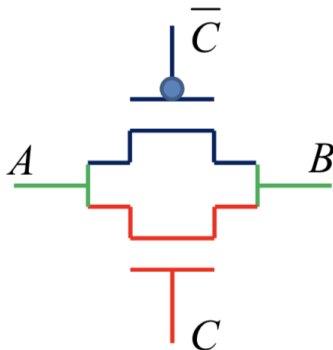
传输门

$V_C = V_{DD}$, V_A 上升到 V_{DD} , V_B 充电:



- Note: NMOS在 $V_{GS} < V_T$ 的时候关断
- 仅用NMOS传输VDD信号时, 会面临阈值压降问题

因此, 为了能让0和1信号都从A传输到B且避免阈值压降问题, 我们可以用NMOS和PMOS结合的方式构建CMOS开关, 也就是**传输门** (Transmission Gate)

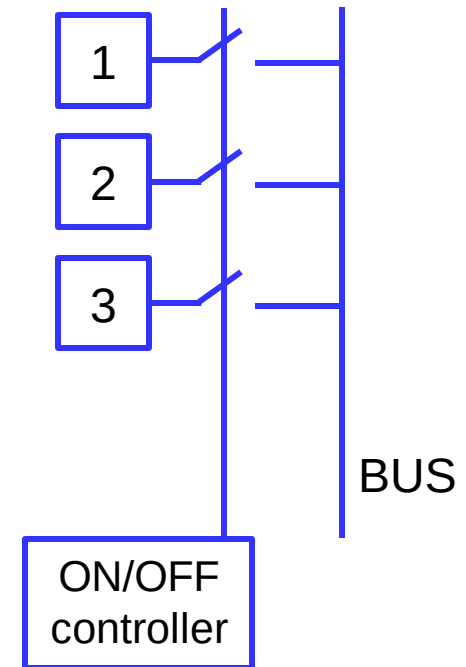
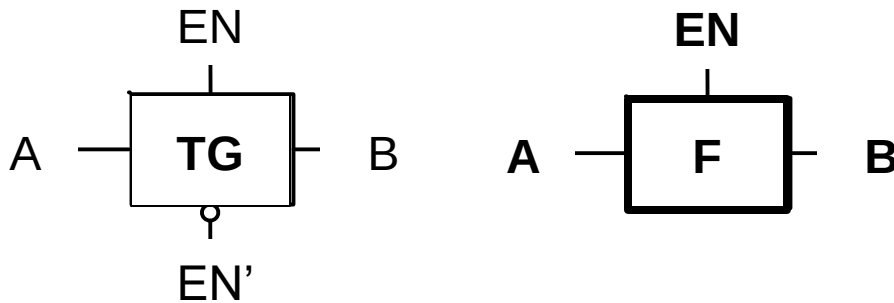


TG symbols

Courtesy:
李国林

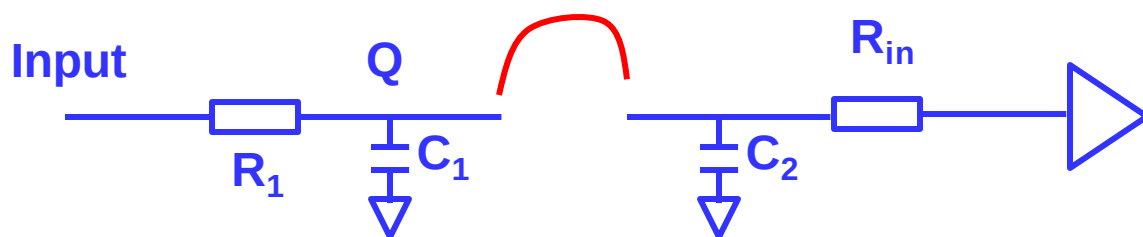
三态门

- 两个控制条件
 - EN为高, TG 打开, $F=A$, 可传输0、1两态
 - EN为低, TG 关闭, F 和输入A隔离, 第三态
- 应用
 - 示例: N-to-1 选择器



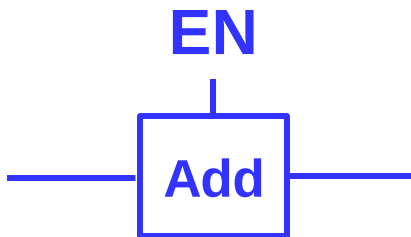
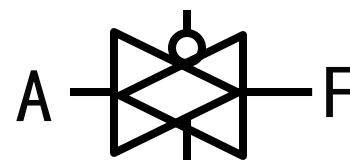
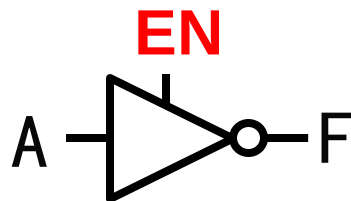
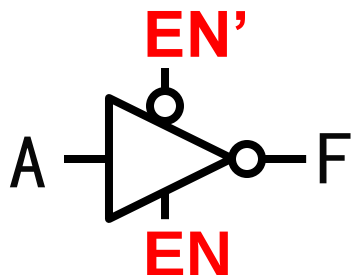
三态门与高阻态

- 三态与三态门
 - 逻辑0, 逻辑1, 高阻态Z
- 高阻态：在电路中相当于悬空(floating)
- 思考：如果用万用表测其到地电压可能是什么结果？



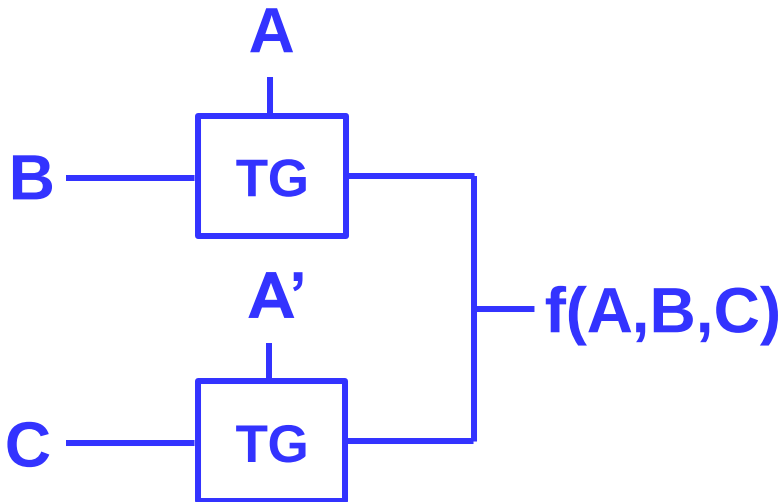
带使能的逻辑门

- 反相器、传输门

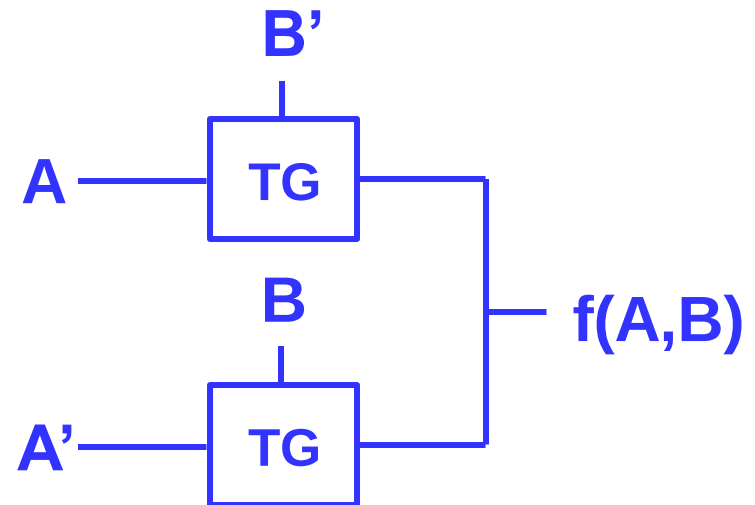


传输门

- $f(A,B,C) = AB + A'C$



- $f(A,B) = \text{XOR}(A,B) = AB' + A'B$



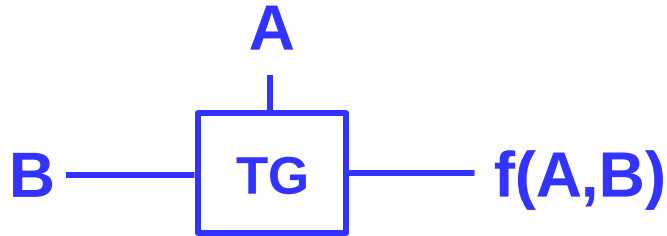
Quiz: Can you swap $A \leftrightarrow B$, $A' \leftrightarrow C$?

No! Please ensure that

- (1) The output is not floating;
- (2) The output has no driving conflict;

Transmission gate

- $f(A,B) = AB$



Note:
Ensure that the output is
not floating;

- Is there a problem?

CMOS 门之外的逻辑

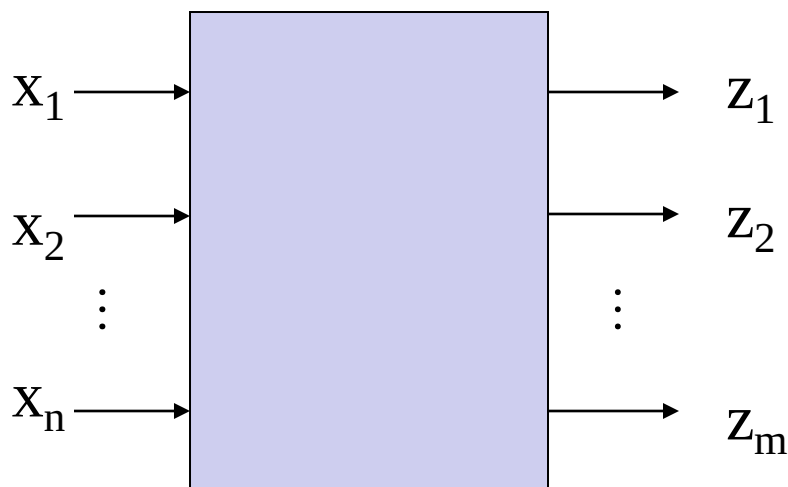
- 传输管逻辑(PTL)
- 伪NMOS
- 电流模逻辑 (CML)

本节课目录

- 从布尔表达式到开关电路
- 定义、分析、设计
- 评价指标和冒险
- 常用组合逻辑模块

组合逻辑的定义

- 输出是输入逻辑变量的函数
- 当前输出仅由**当前**输入变量及延时决定
 - 无记忆



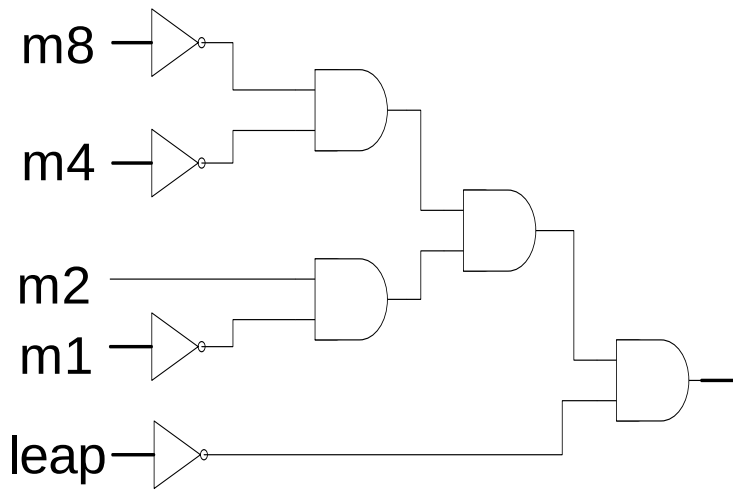
$$\begin{cases} Z_1 = f_1(x_1, x_2, \dots, x_n) \\ Z_2 = f_2(x_1, x_2, \dots, x_n) \\ \vdots \\ \vdots \\ Z_m = f_m(x_1, x_2, \dots, x_n) \end{cases}$$

思考：可以有反馈环路吗？**不可以！**

组合逻辑电路的分析方法

- 目标

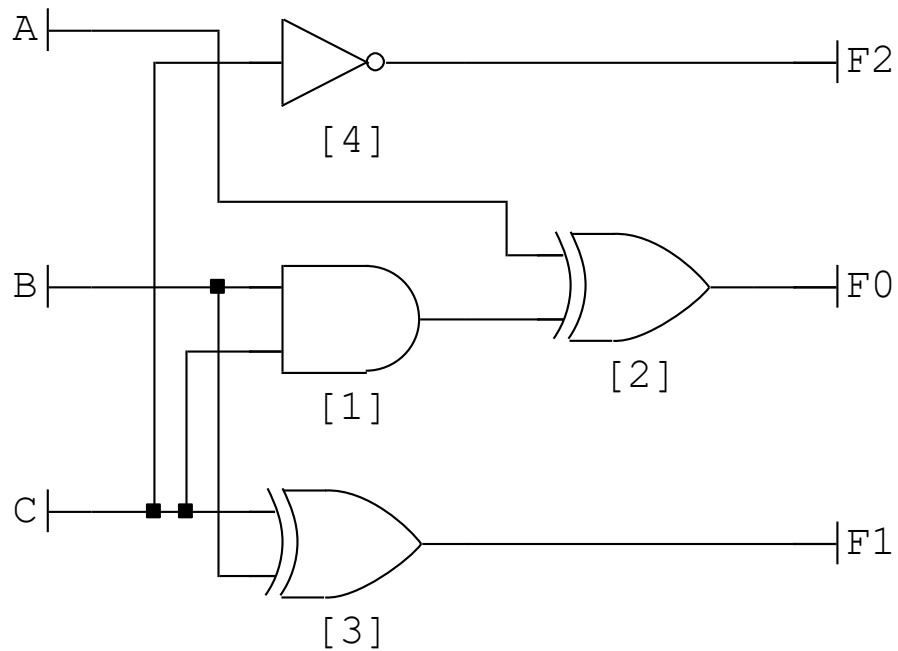
- 给定设计的电路（晶体管/逻辑门）
- 在真值表，布尔表达式等形式中表达函数
- 低层次 到 高层次 分析



示例 1

$$d28 = a_8' \text{ and } a_4' \text{ and } a_2 \text{ and } a_1' \text{ and } \text{leap}'$$

例2



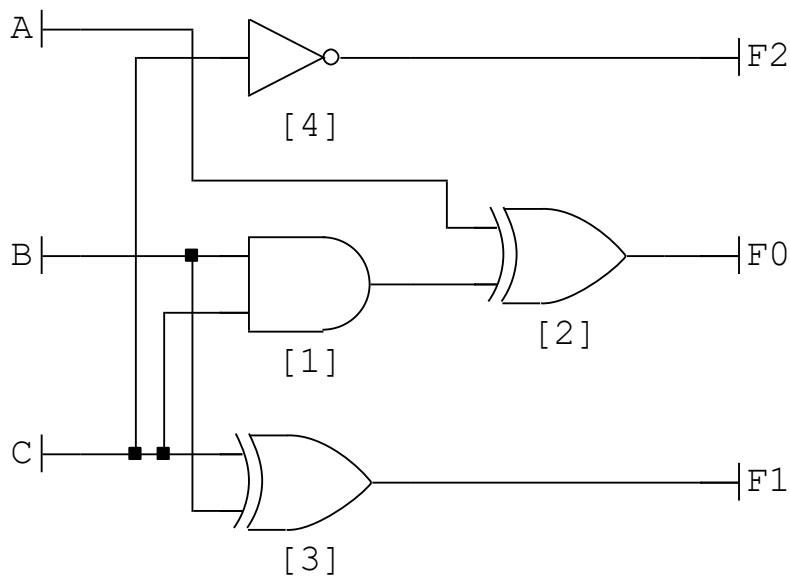
输入：{A B C}

输出：{F0 F1 F2}

电路功能是什么？

例2

- 列出布尔表达式与真值表



$$F_0 = A \oplus (BC)$$

$$F_1 = B \oplus C$$

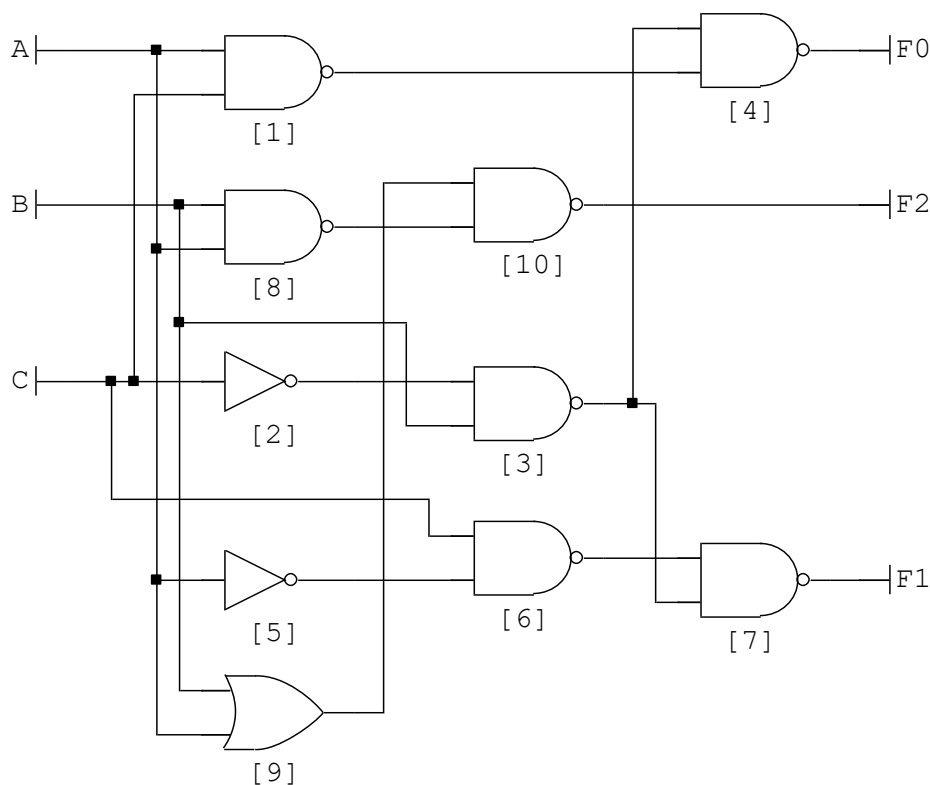
$$F_2 = C'$$

A	B	C	F0	F1	F2
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

$$(F_0 F_1 F_2) = (ABC) + 1$$

例3

- 真值表如右图所示，该电路功能是？

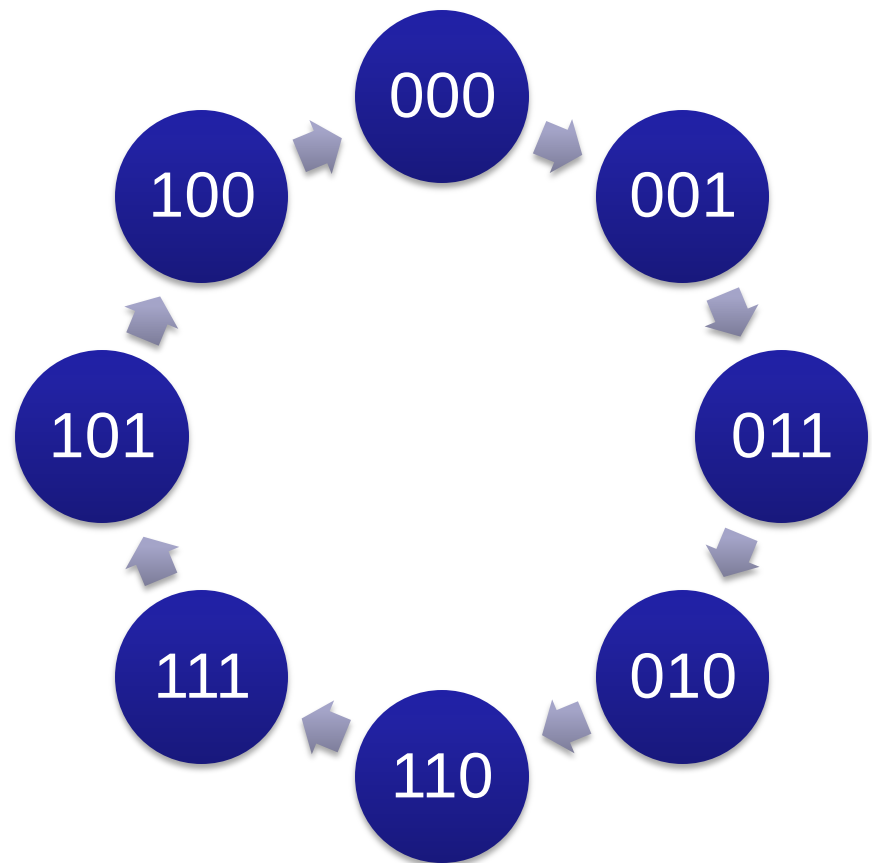


A	B	C	F0	F1	F2
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	1	1	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	1

例3

- 真值表重排顺序后
 - 格雷码递增编码器

A	B	C	F0	F1	F2
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0



组合逻辑电路的设计/综合过程

高层次到低层次

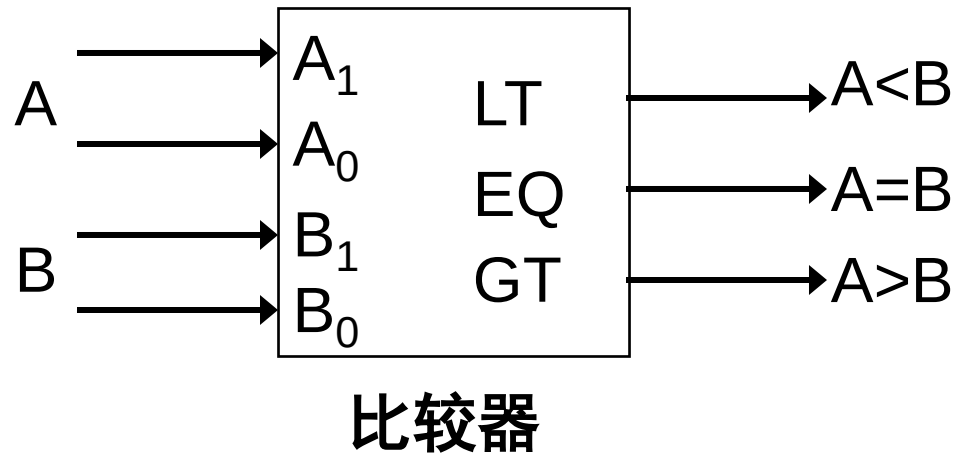
抽象计算（算法）→电路结构→基础器件

- 逻辑抽象（算法）
- 真值表 -> 化简的逻辑
- 基础器件 -> 逻辑电路结构

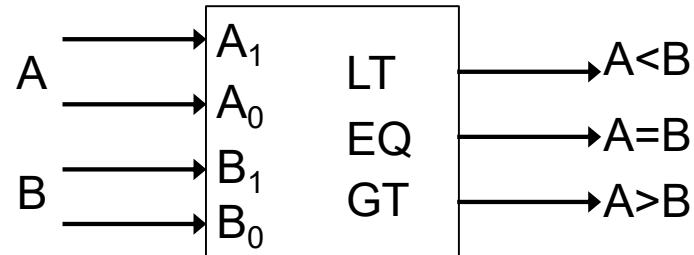
逻辑抽象

- 一个比较器设计

- 功能: 比较两个2比特二进制数的大小
- 输入: A_1A_0 和 B_1B_0
- 输出: $LT(A<B)$, $EQ(A=B)$, $GT(A>B)$



列出真值表



比较器

Inputs

Outputs

A ₁	A ₀	B ₁	B ₀	LT	EQ	GT
1	0	0	0	0	0	1
			1	0	0	1
		1	0	0	1	0
			1	1	0	0
1	1	0	0	0	0	1
			1	0	0	1
		1	0	0	0	1
			1	0	1	0

用卡诺图化简

A_1	A_0	B_1	B_0	LT	EQ	GT
1	0	0	0	0	0	1
			1	0	0	1
		1	0	0	1	0
			1	1	0	0
1	1	0	0	0	0	1
			1	0	0	1
		1	0	0	0	1
			1	0	1	0

B_1B_0 $A_1A_0 \backslash$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

B_1B_0 $A_1A_0 \backslash$	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

$$LT=(A<B)=$$

$$\bar{A}_1B_1 + \bar{A}_1\bar{A}_0B_0 + \bar{A}_0B_1B_0$$

布尔表达式

$$LT=(A<B) = \bar{A}_1 B_1 + \bar{A}_1 \bar{A}_0 B_0 + \bar{A}_0 B_1 B_0$$

2级与或形式

$$\begin{aligned} EQ=(A=B) &= \bar{A}_1 \bar{A}_0 \bar{B}_1 \bar{B}_0 + \bar{A}_1 A_0 \bar{B}_1 B_0 + A_1 \bar{A}_0 B_1 \bar{B}_0 + A_1 A_0 B_1 B_0 \\ &= \bar{A}_1 \bar{B}_1 (\bar{A}_0 \bar{B}_0 + A_0 B_0) + A_1 B_1 (\bar{A}_0 \bar{B}_0 + A_0 B_0) \\ &= \bar{A}_1 \bar{B}_1 \overline{(A_0 \oplus B_0)} + A_1 B_1 \overline{(A_0 \oplus B_0)} \\ &= (\bar{A}_1 \bar{B}_1 + A_1 B_1) \overline{(A_0 \oplus B_0)} = \overline{(A_1 \oplus B_1)} \bullet \overline{(A_0 \oplus B_0)} \end{aligned}$$

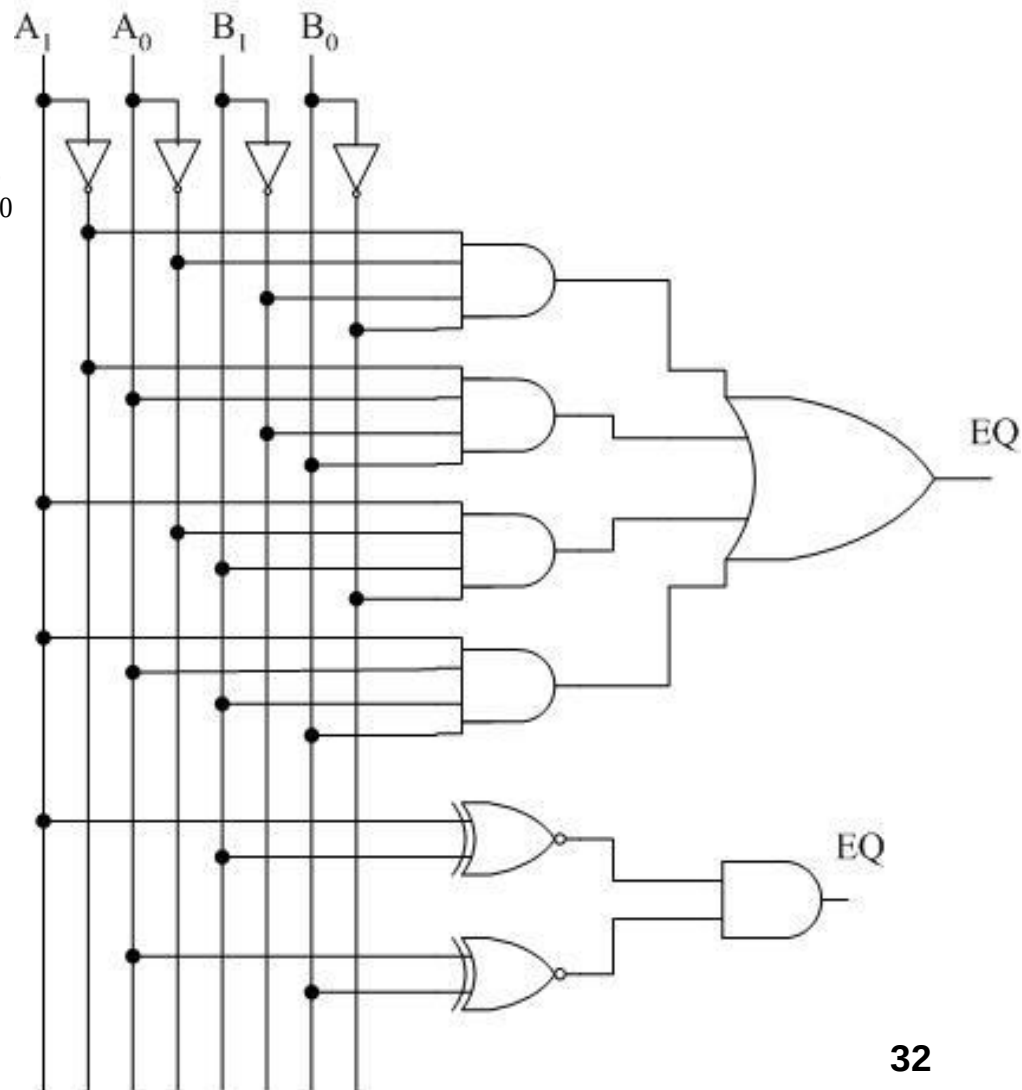
$$GT=(A>B)=A_1 \bar{B}_1 + A_0 \bar{B}_1 \bar{B}_0 + A_1 A_0 \bar{B}_0$$

逻辑门/电路

EQ=

$$\bar{A}_1\bar{A}_0\bar{B}_1\bar{B}_0 + \bar{A}_1A_0\bar{B}_1B_0 + A_1\bar{A}_0B_1\bar{B}_0 + A_1A_0B_1B_0$$

$$= \overline{(A_1 \oplus B_1)} \bullet \overline{(A_0 \oplus B_0)}$$



看起来很容易？

- 开放式讨论
 - 50个输入的真值表或卡诺图？
 - 如何处理复杂逻辑？
 - 从2级到多级(BDD?)
 - $f(A,B,\dots)=A \cdot g(B,C,\dots)+A' \cdot h(B,C,\dots)$
 - 分而治之
 - $f(A,B,\dots) = f(g(\dots), h(\dots), \dots)$
 - 结构化、复用

处理复杂逻辑

- 中间逻辑和共享项

- 例1: $F = abc + abd + a'c'd' + b'c'd'$

- 使 $X = ab, Y = c + d$, 则 $F = XY + X'Y'$

- 例2: $X = ab(c(d + e) + f + g) + h, Y = ai(c(d + e) + f + j) + k$

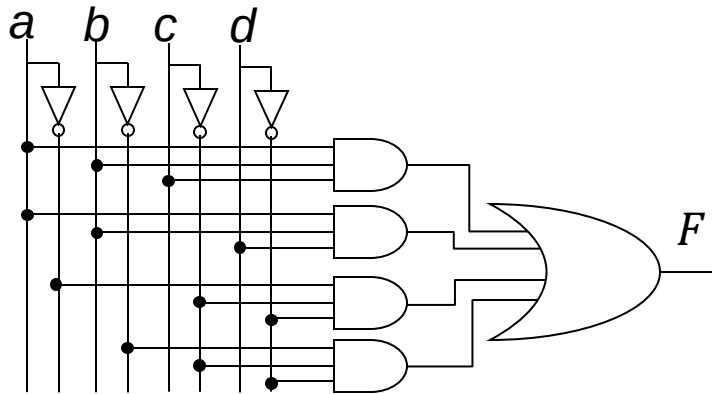
- 使 $N = a(c(d + e) + f)$, 则

- $$X = b(N + ag) + h, Y = i(N + aj) + k$$

设计优化

设计者可能有不同的功耗/性能/面积要求

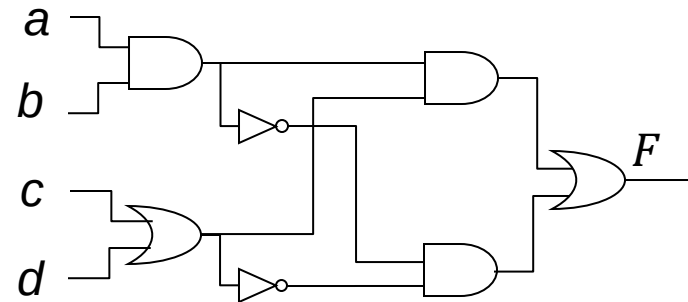
$$F = abc + abd + a'c'd' + b'c'd'$$



4xNOT, 4xAND3,
1xOR4

$$X = ab, Y = c + d$$

$$F = XY + X'Y'$$

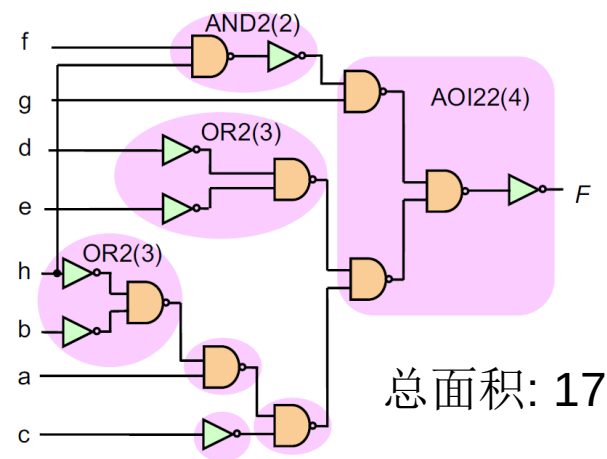
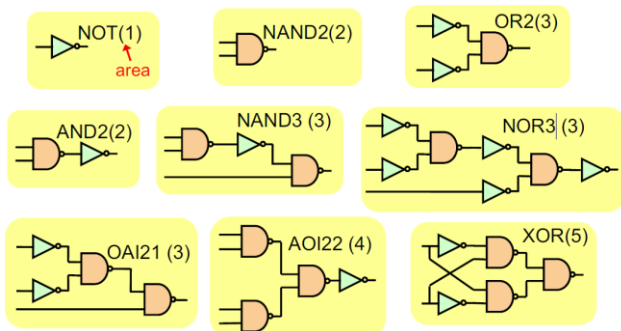
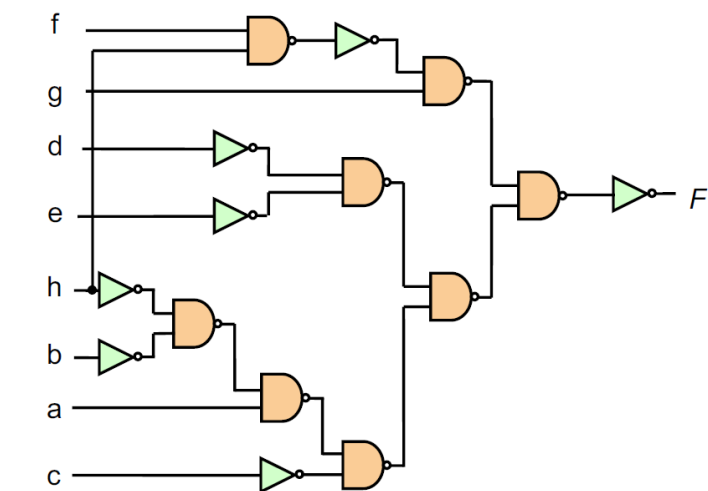


2xNOT, 3xAND2
2xOR2

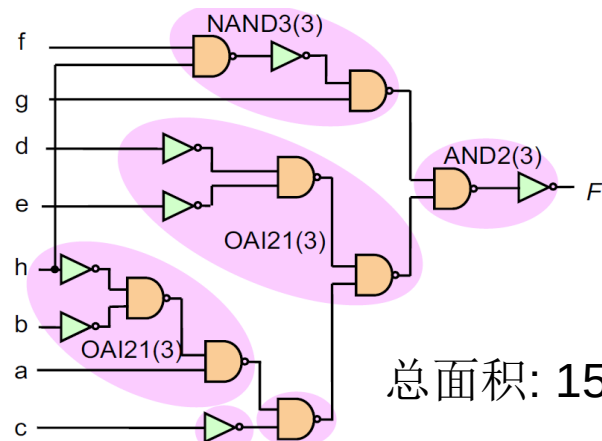
设计优化

- 用工艺库基本单元覆盖组合逻辑

- 典型的EDA优化问题

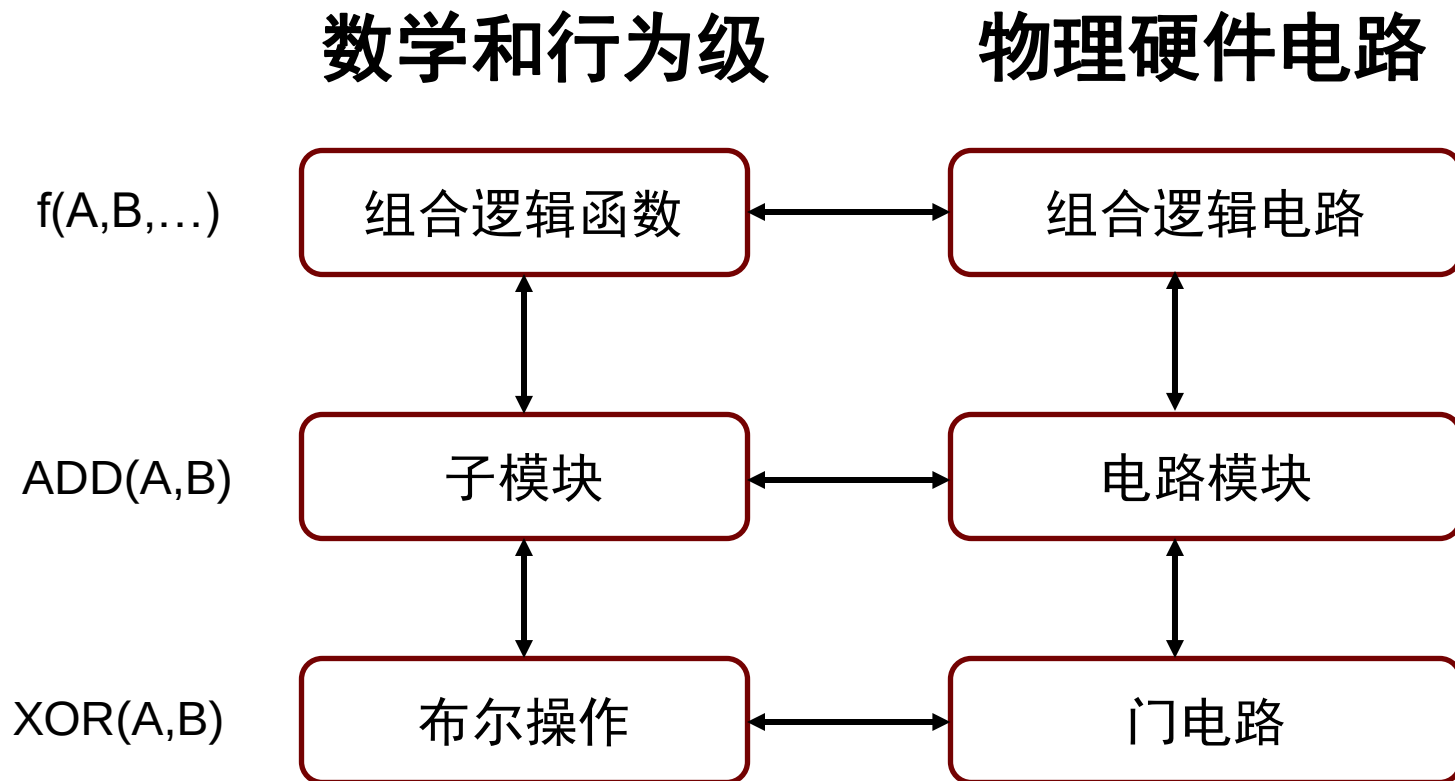


总面积: 17



总面积: 15

组合逻辑和电路



本节课目录

- 从布尔表达式到开关电路
- 定义、分析、设计
- 评价指标和冒险
- 常用组合逻辑模块

评价逻辑电路的主要指标

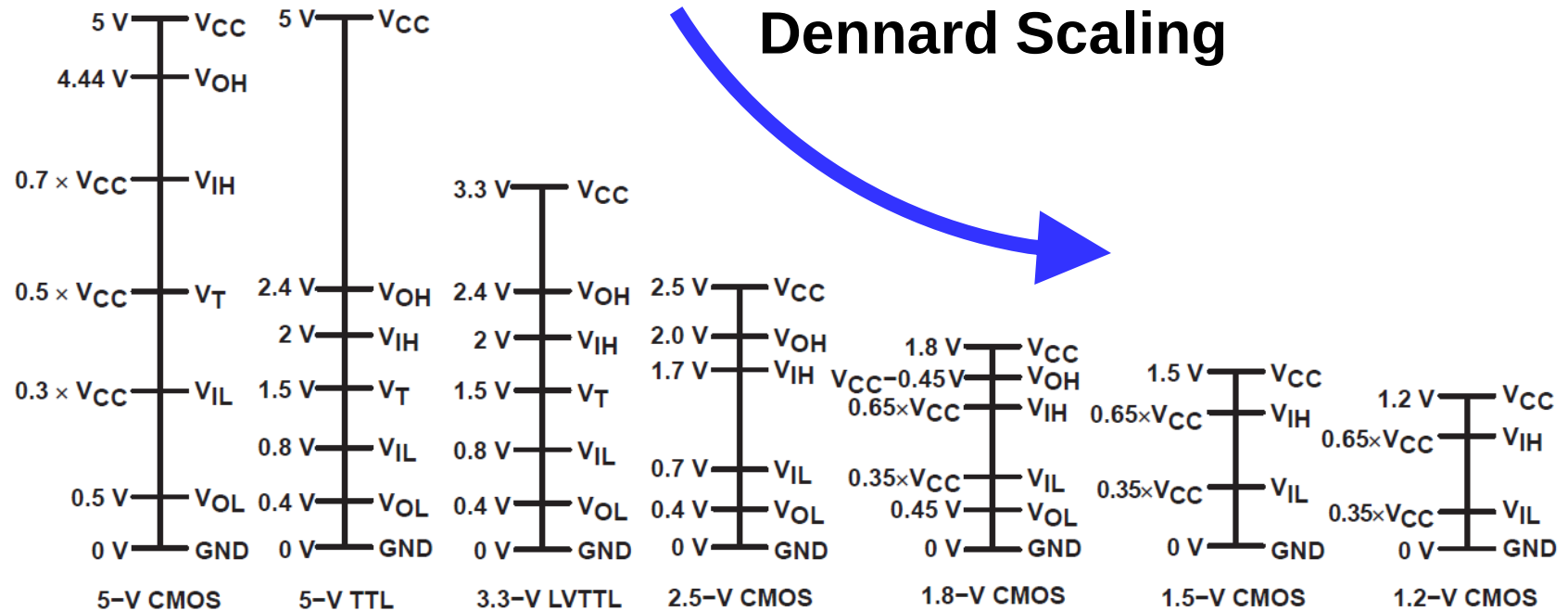
- 稳态因素

- Logic voltage levels 逻辑电平
- DC noise margins 噪声容限
- Area 面积（前面介绍过）
- Fan-out 扇出系数/驱动负载能力

- 动态因素

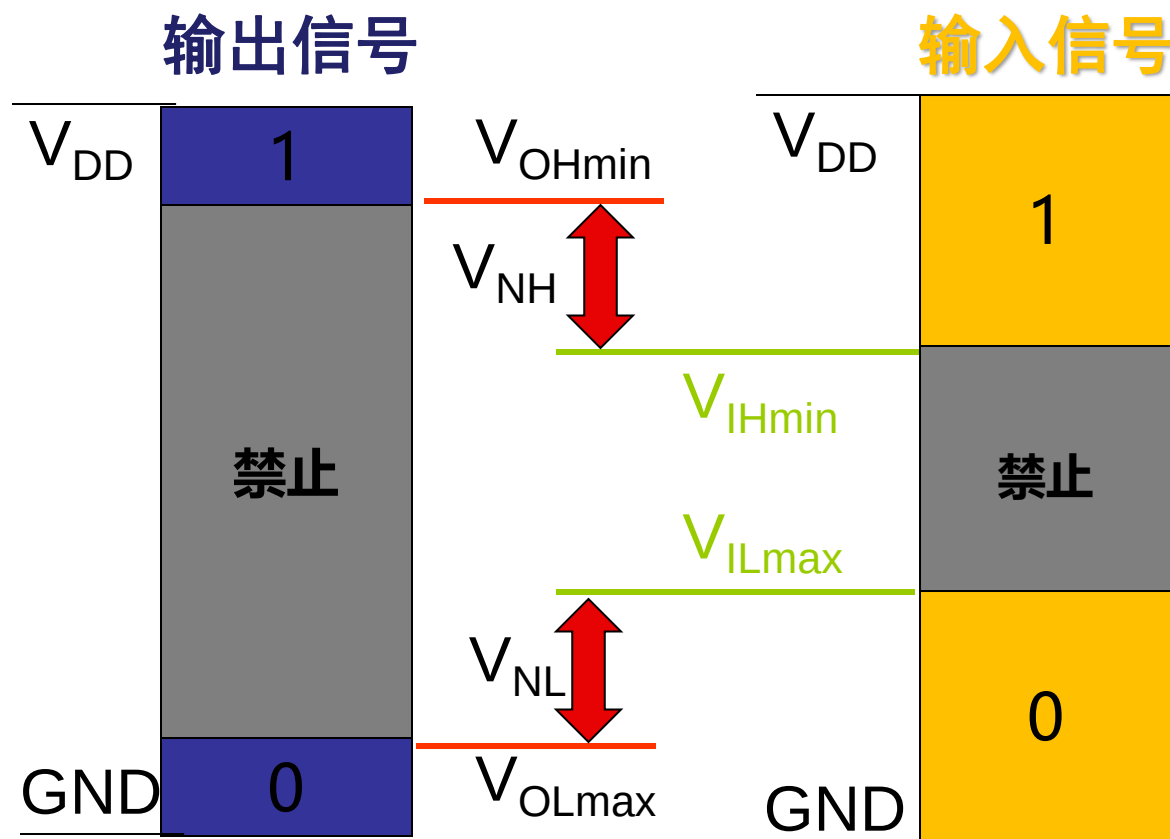
- Speed/Delay 速度/延时
- Power Dissipation 功耗
- Noise 噪声(不讲)

逻辑电平“家族”



I/O does not scale much

数字电路：噪声容限



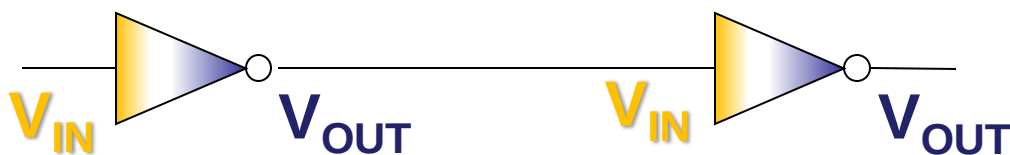
噪声容限：

$$V_{NH} = V_{OHmin} - V_{IHmin}$$

$$V_{NL} = V_{ILmax} - V_{OLmax}$$

亮点：

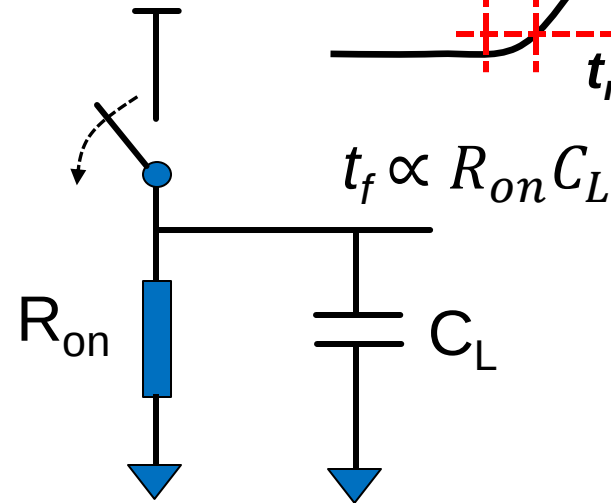
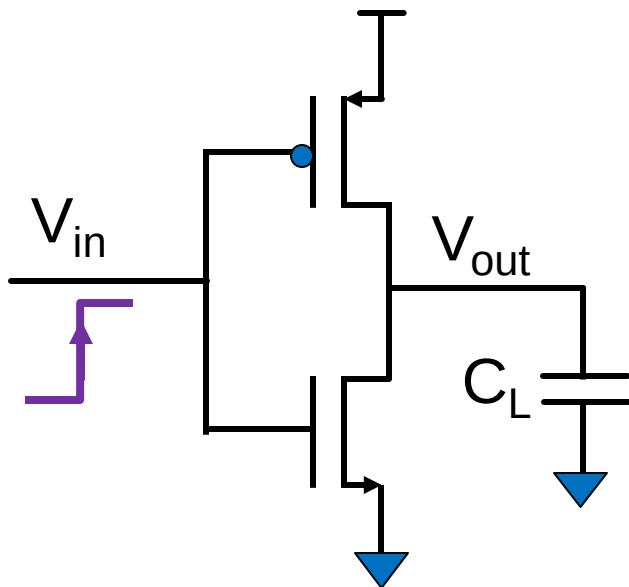
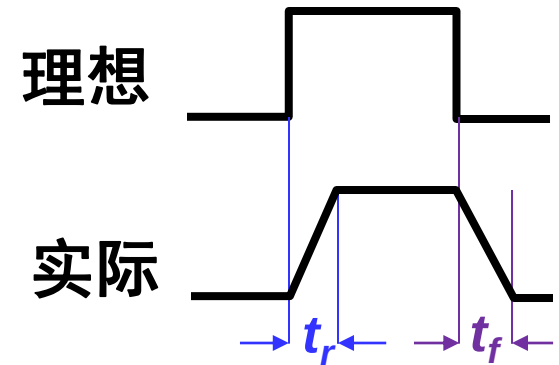
只要输入信号不超过边界，就没有误判行为！



速度：转换时间

- 输出信号

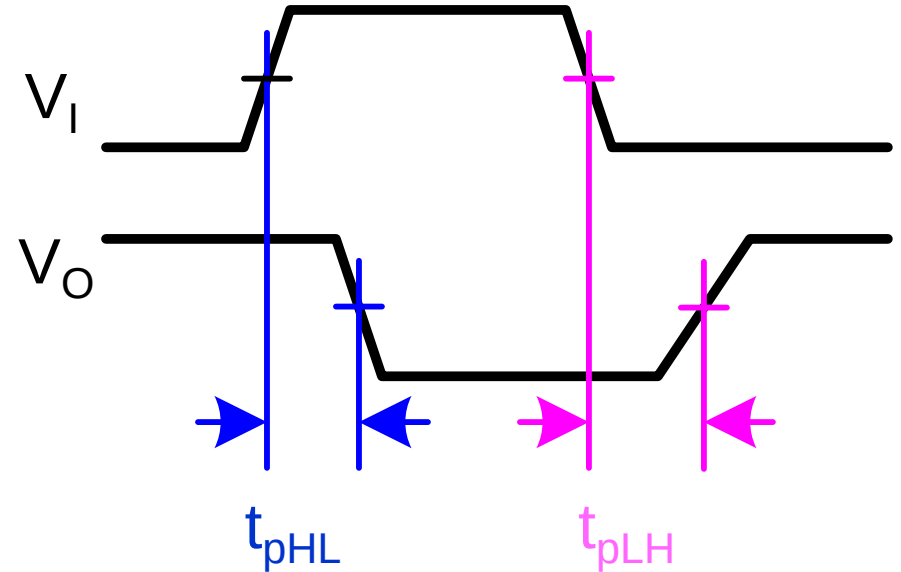
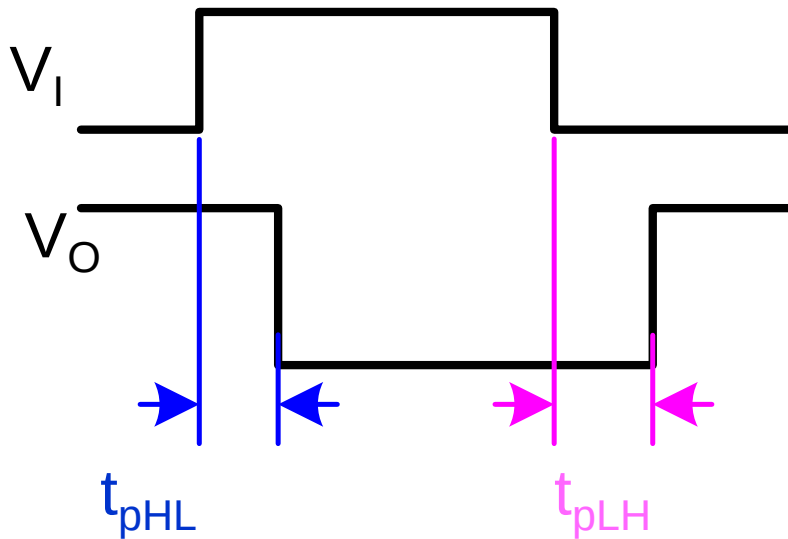
- 上升时间 t_r 和下降时间 t_f
- 实际使用中
 - 10%-90% 或 20%-80% VDD



速度：延时

- 传播延时

- 从输入到输出



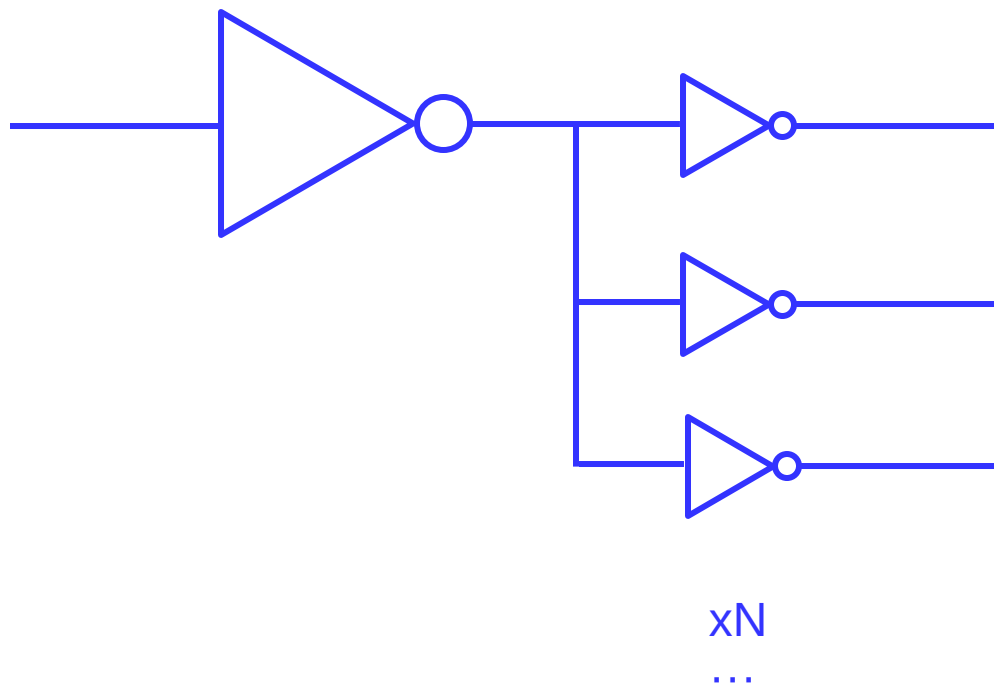
平均值 $t_{pd} = (t_{pHL} + t_{pLH}) / 2$

最大值 $\max(t_{pHL}, t_{pLH})$, 最小值 $\min(t_{pHL}, t_{pLH})$

扇出系数

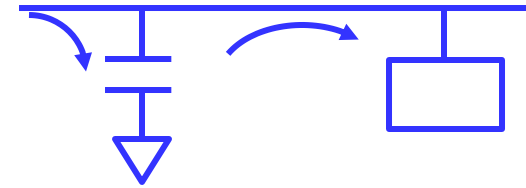
Fan out

- 在最坏负载情况下，一个逻辑门能驱动的输入端数目



功率和能量

- 功率和能量消耗
- 瞬时功率 vs 平均功率



$$P(t) = i_{DD}(t)V_{DD}$$

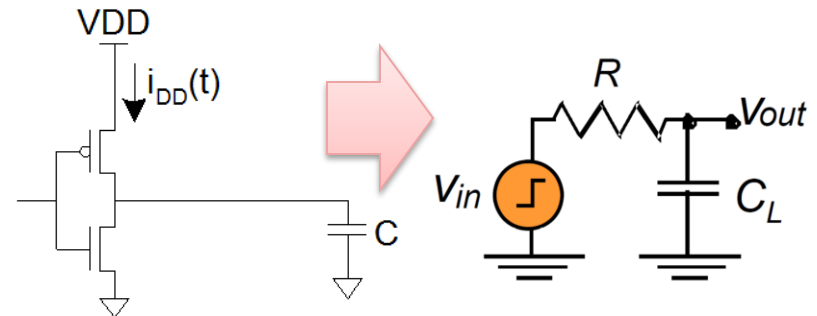
$$E = \int_0^T P(t)dt = \int_0^T i_{DD}(t)V_{DD}dt$$

$$P_{\text{avg}} = \frac{E}{T} = \frac{1}{T} \int_0^T i_{DD}(t)V_{DD}dt$$

CMOS功耗模型

- 动态功耗 P_{dynamic}
 - 电容充放电
- 减少动态功耗
 - 降低电源电压和电容
 - 降低频率和翻转概率

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{dynamic_short}} + P_{\text{leakage}}$$



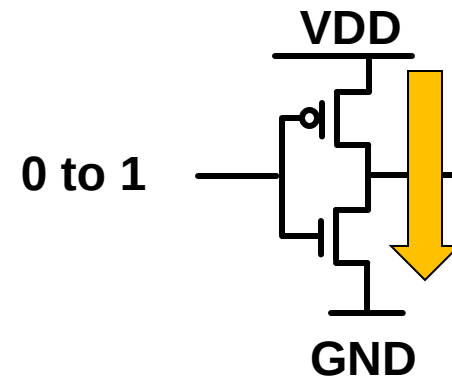
$$P = \frac{E}{\Delta T} = E_{0-1} f_{0-1} = C_L V_{DD}^2 f_{0-1}$$

活动因子

$$= C_L V_{DD}^2 \alpha_{0-1} f$$

工作频率
翻转概率

- 动态功耗 $P_{\text{dynamic_short}}$
 - 上拉电路和下拉电路同时导通
- 功耗延时积(PDP)、能耗延时积(EDP)

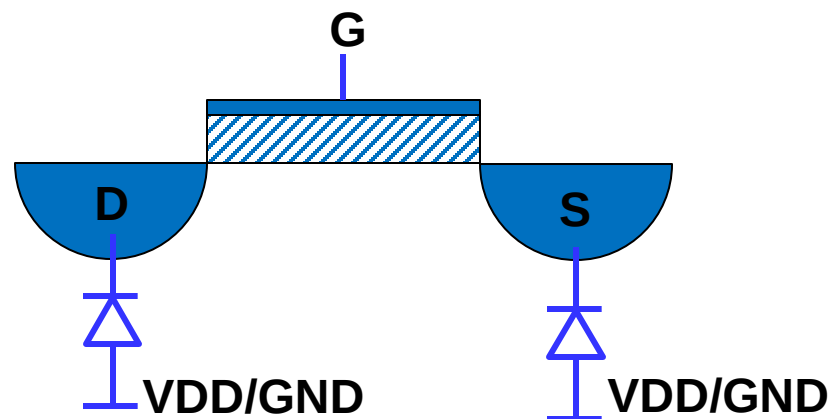
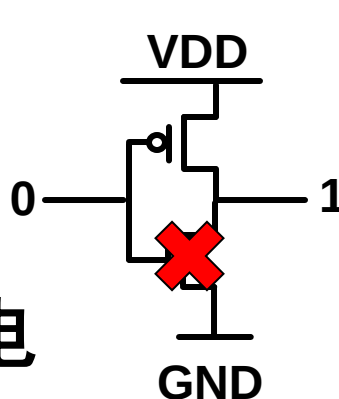


CMOS功耗模型

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{dynamic_short}} + P_{\text{leakage}}$$

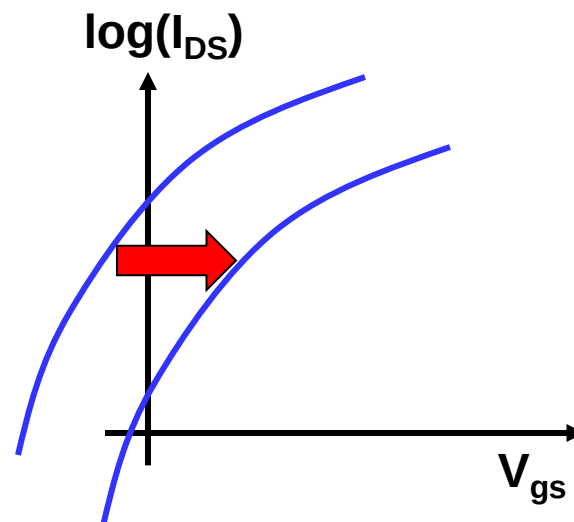
- 静态功耗

- 亚阈值漏电
- 栅极漏电
- D/S 衬底漏电

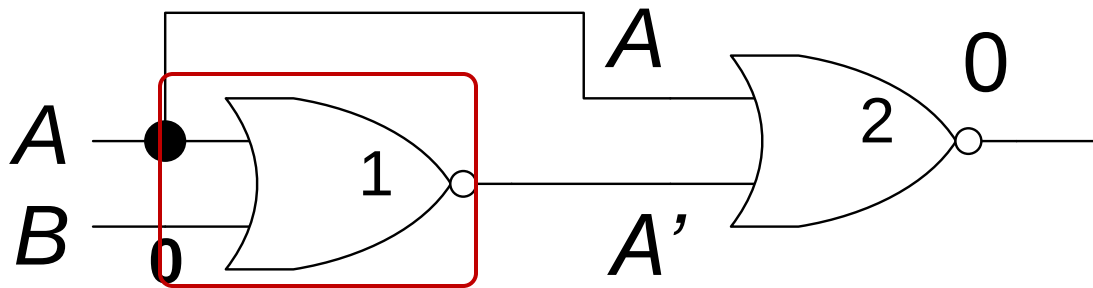


- 减少静态功耗

- 提高阈值电压 $|V_{\text{TH}}|$
- 降低电源电压
- 其他



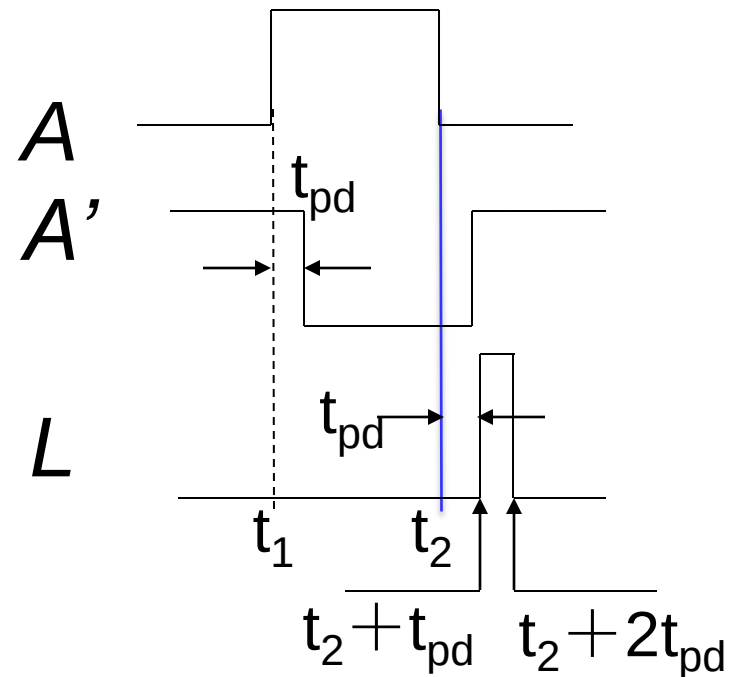
冒险：举例



$$L = ((A+B)' + A)'$$

理想地, $B = 0$

$$\rightarrow L = (A' + A)' = 0$$



毛刺与冒险

- Glitch 毛刺

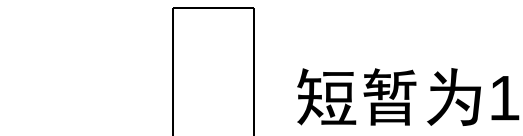
- 在组合逻辑电路输出端的多余脉冲，本应保持不变的有瞬时变化

- 静态冒险：一个本应保持不变的输出经历瞬时转换

- 静态1的冒险：取值为1的输出瞬时经历0状态
- 静态0的冒险：取值为0的输出瞬时经历1状态

- 动态冒险（非本课程讨论范围）

- 本应单次跳变的输出信号发生不止一次的跳变

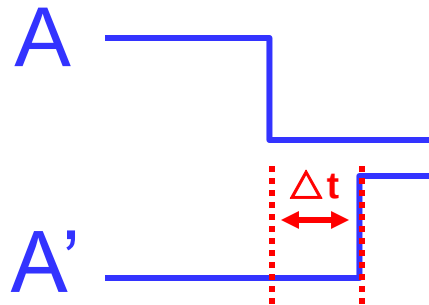
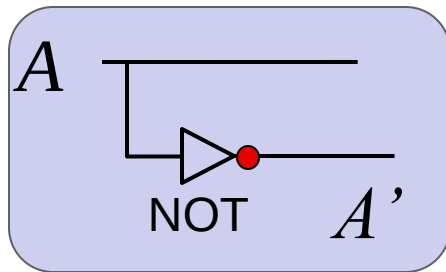


静态0的冒险



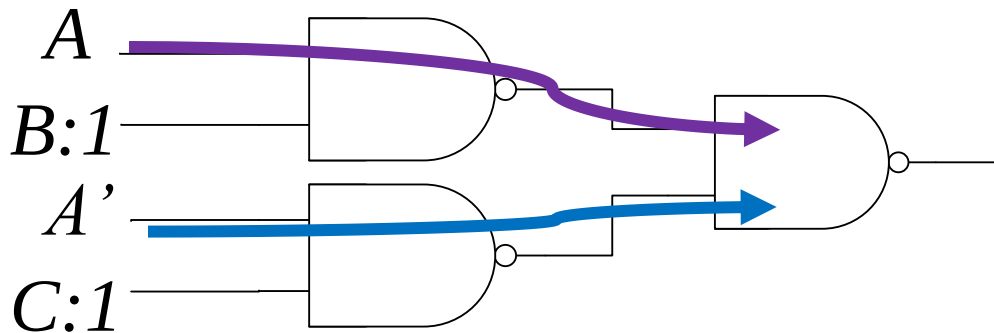
静态1的冒险

静态1的冒险: 示例



$$L = AB + \bar{A}C$$

$$B = C = 1 \rightarrow L = A + A' = 1$$



$B=C=1$, 当 $A: 1 \rightarrow 0$ 时, 由于 A 和 A' 变化时刻存在差异, A 先变为 0, A' 还未变成 1 (仍然为 0), 导致本应为 1 的 $L = A + A' = 0$, 存在 1 冒险!

静态1冒险产生原因分析

BC \ A	00	01	11	10
0	0	1	1	0
1	0	0	1	1

$$L = AB + \bar{A}C$$

输入初始值011

输入最终值111

BC

- 如输入初始值和输入最终值不能被同一本原蕴含项覆盖，则该本原蕴含项内元素间状态切换可能出现毛刺

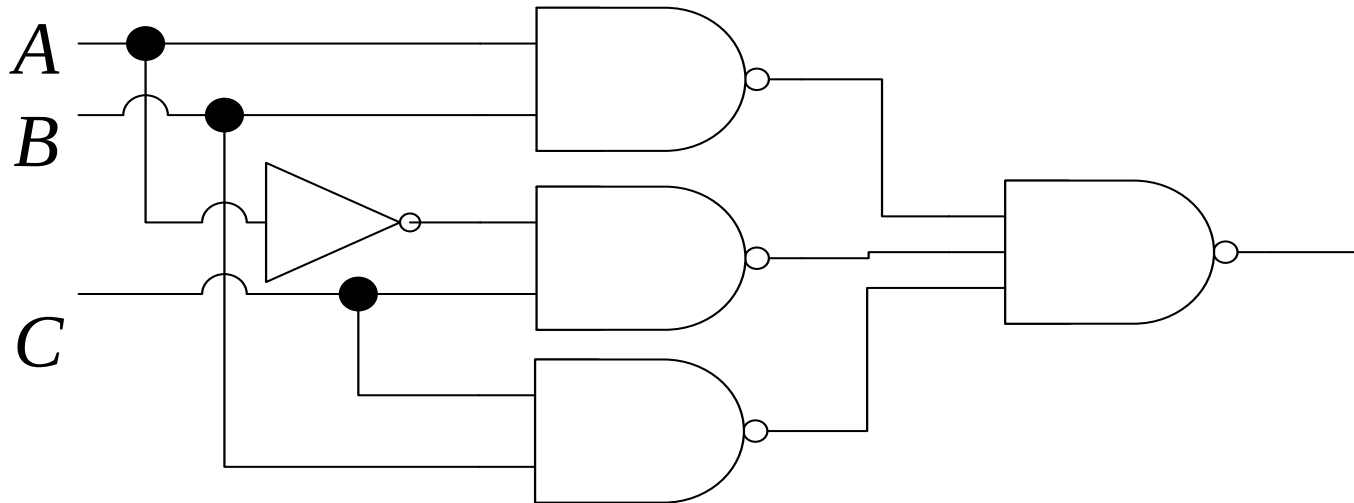
上面的例子加入BC项

$$L = AB + \bar{A}C + BC$$

当B=1, C=1时, $ABC \leftrightarrow A'BC$
不引起输出变化

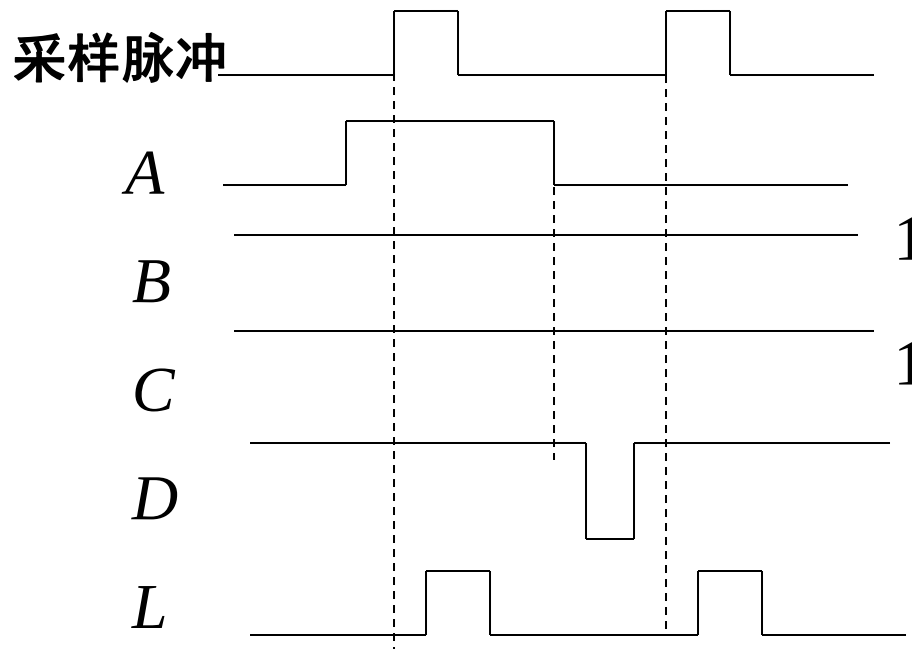
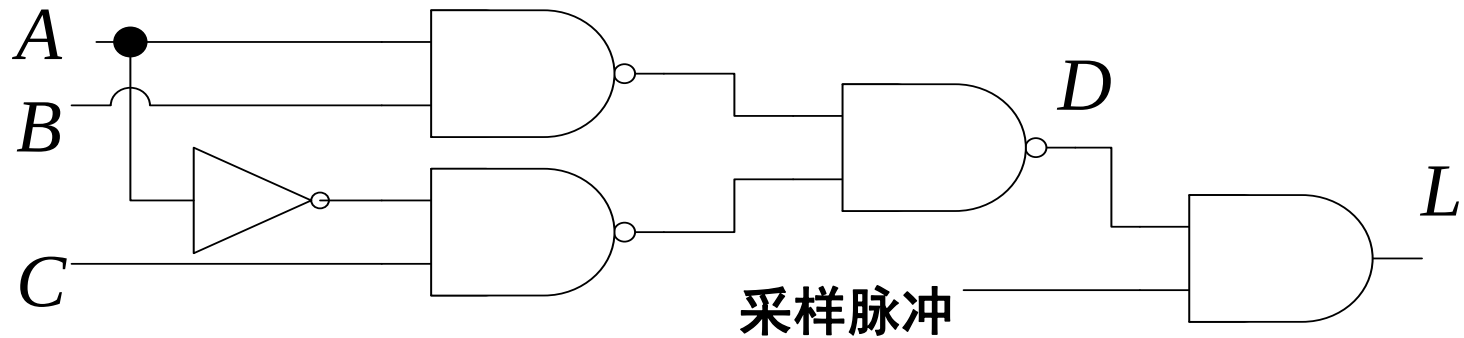
无冒险电路

- 增加适当的冗余蕴含项可以消除冒险



$$D = AB + \bar{A}C + BC$$

利用采样脉冲来消除冒险



采样脉冲在D
稳定后采样

本节课目录

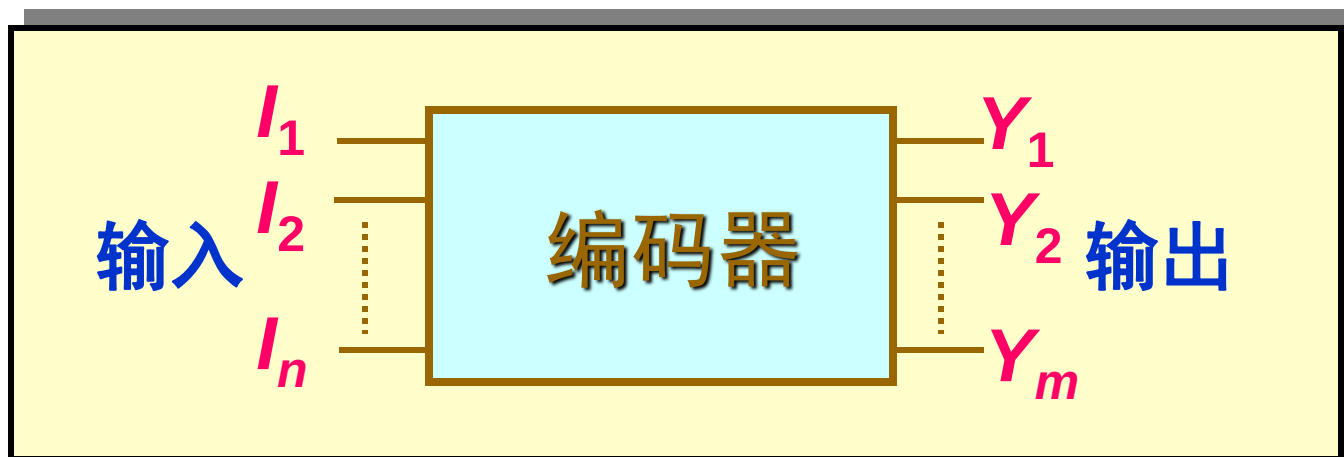
- 从布尔表达式到开关电路
- 定义、分析、设计
- 评价指标和冒险
- 常用组合逻辑模块

常用组合逻辑

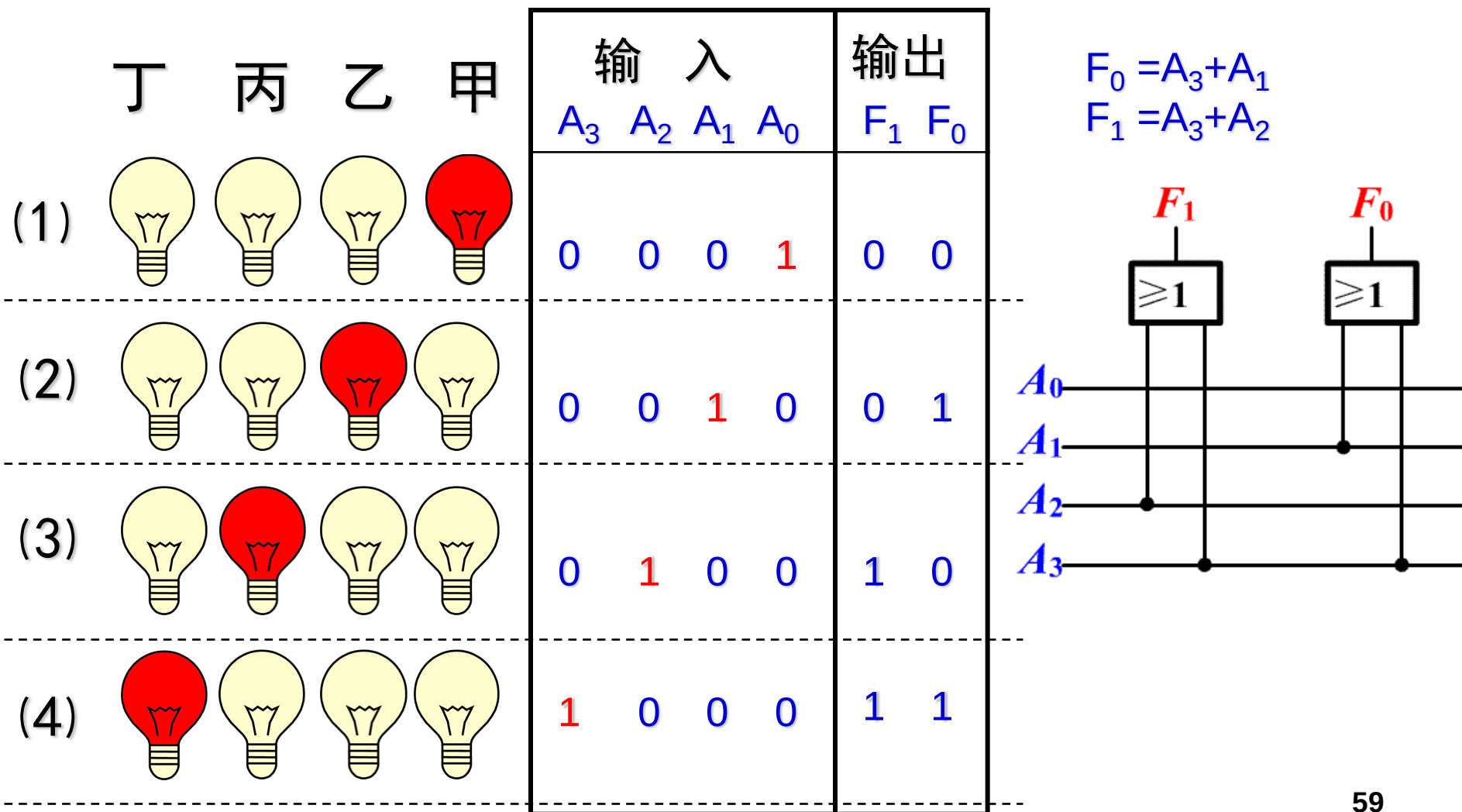
- 编码器 encoder / 译码器 decoder
- 多路选择器 multiplexer / mux
- 加法器 adder

编码器Encoder

二进制编码器：用 m 位二进制代码对 $n \leq 2^m$ 个信号进行编码的电路



4-2线编码器的实现

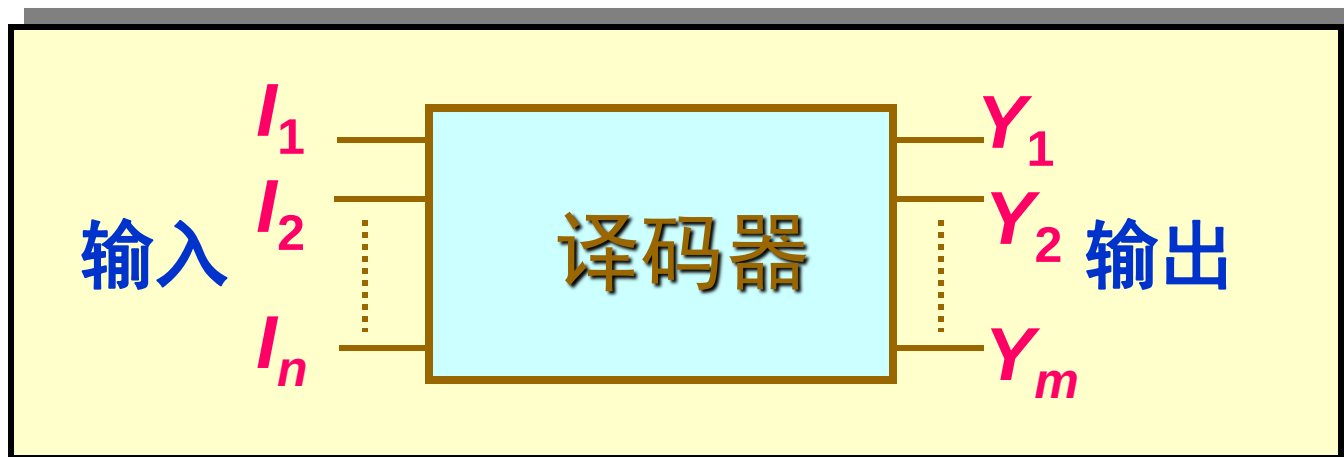


思考

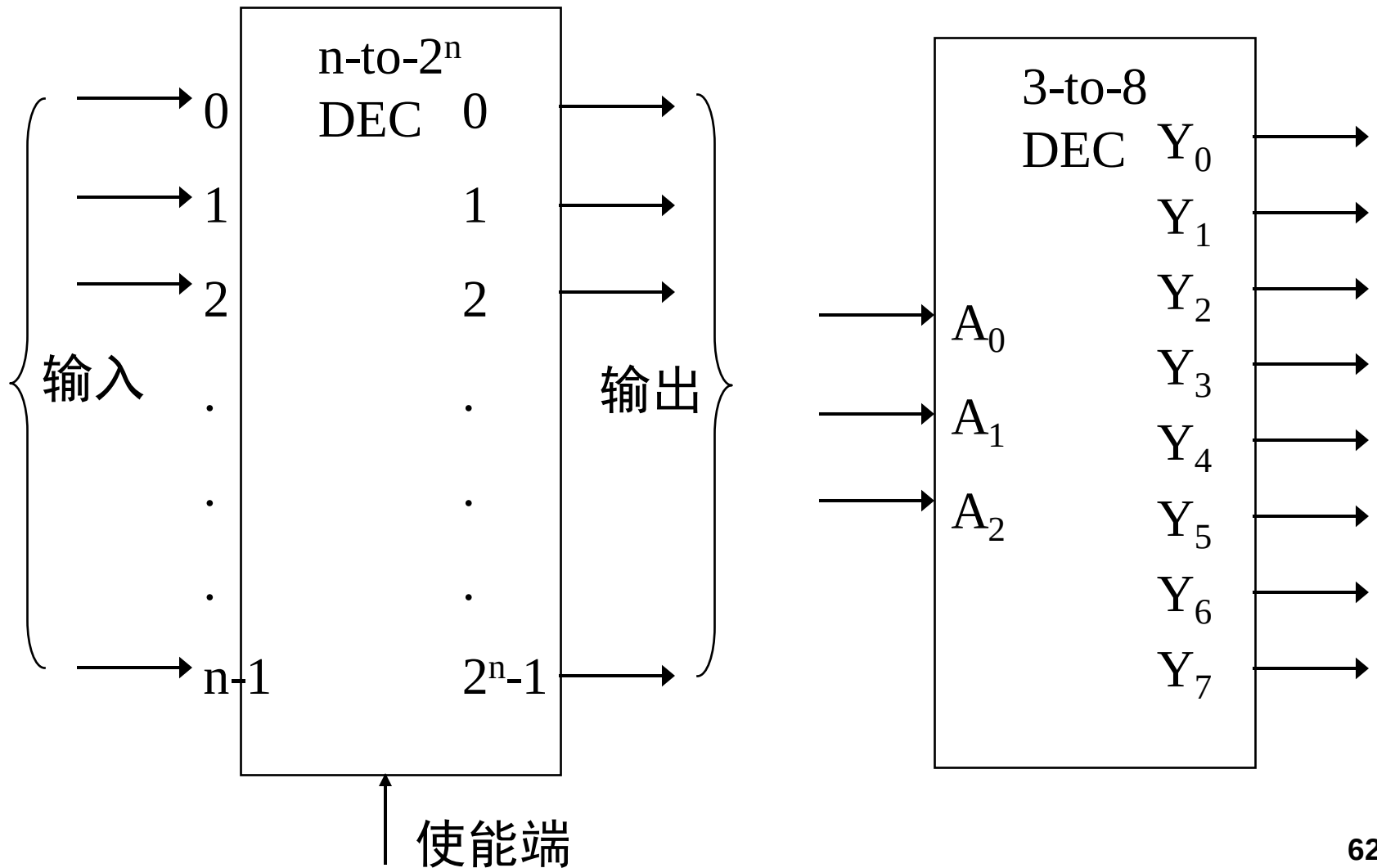
- 如果同时不止一个响应怎么办？
 - 普通编码器 vs 优先编码器

译码器 (Decoder)

- 编码的逆过程
- n - 2^n 线译码器： n 个输入， 2^n 个输出，



n - 2^n 线译码器



设计一种3-8译码器

A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

写出布尔表达式

$$Y_0 = \bar{A}_2 \bar{A}_1 \bar{A}_0 = m_0$$

$$Y_1 = \bar{A}_2 \bar{A}_1 A_0 = m_1$$

$$Y_2 = \bar{A}_2 A_1 \bar{A}_0 = m_2$$

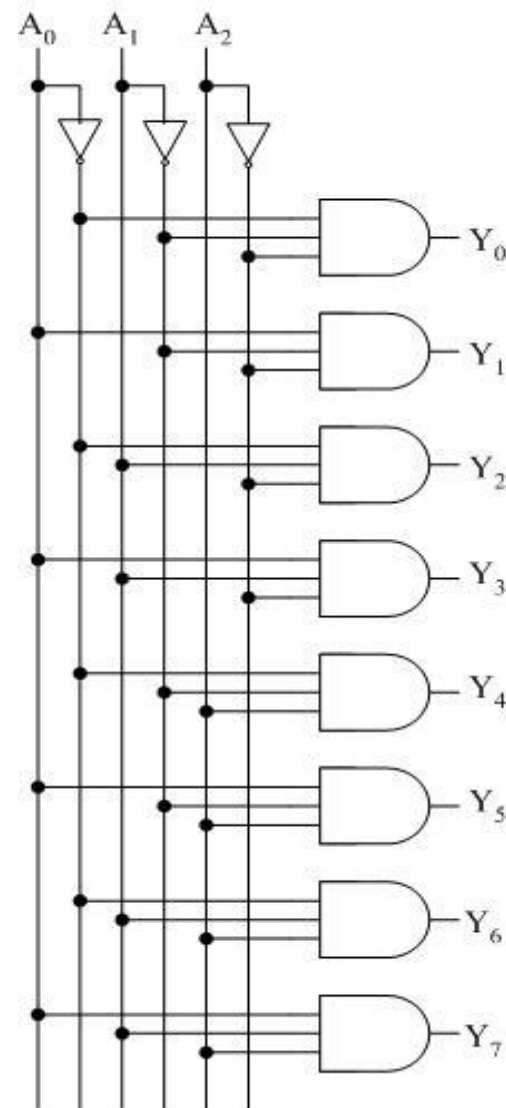
$$Y_3 = \bar{A}_2 A_1 A_0 = m_3$$

$$Y_4 = A_2 \bar{A}_1 \bar{A}_0 = m_4$$

$$Y_5 = A_2 \bar{A}_1 A_0 = m_5$$

$$Y_6 = A_2 A_1 \bar{A}_0 = m_6$$

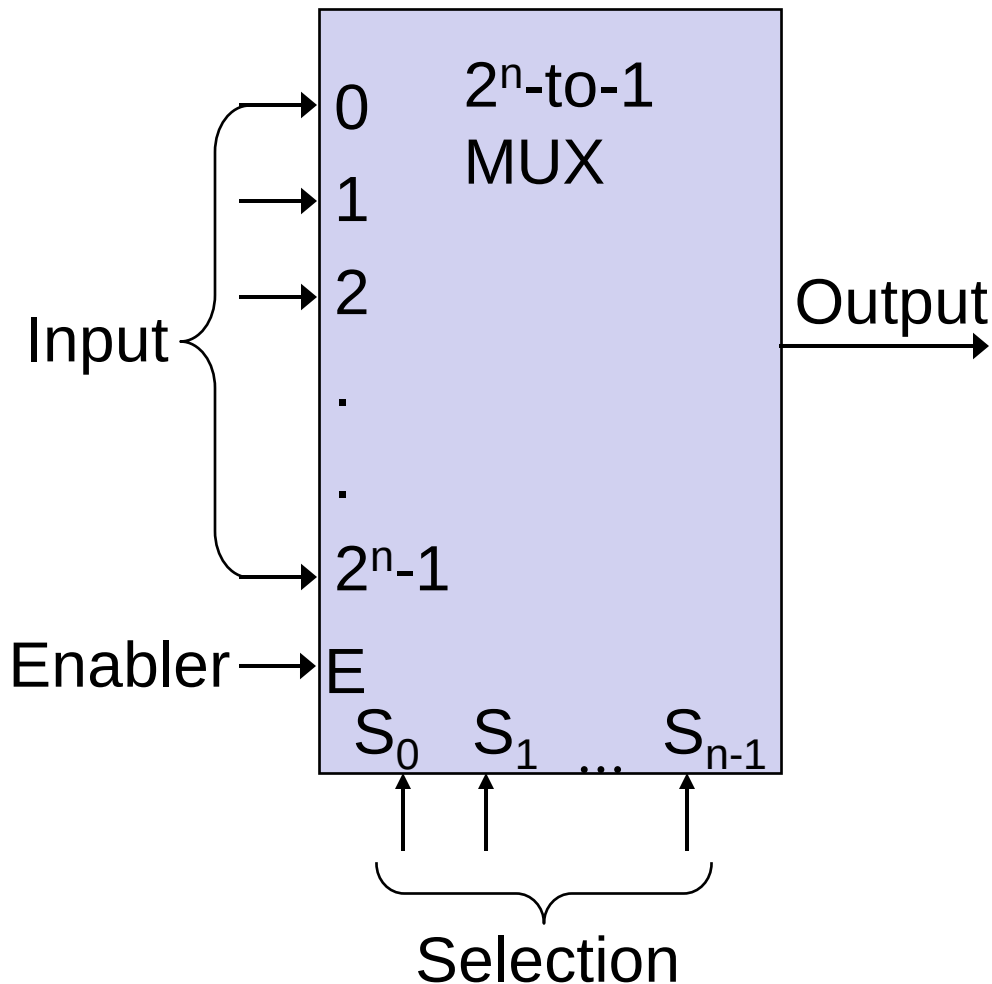
$$Y_7 = A_2 A_1 A_0 = m_7$$





多路选择器Multiplexer

- 从多个输入数据中选一个作为输出



- 8 input signals
 - $D_7D_6D_5D_4D_3D_2D_1D_0$
- 3 input signals
 - Address: $A_2A_1A_0$
- 1 output
 - $Q = D_{A_2A_1A_0}$

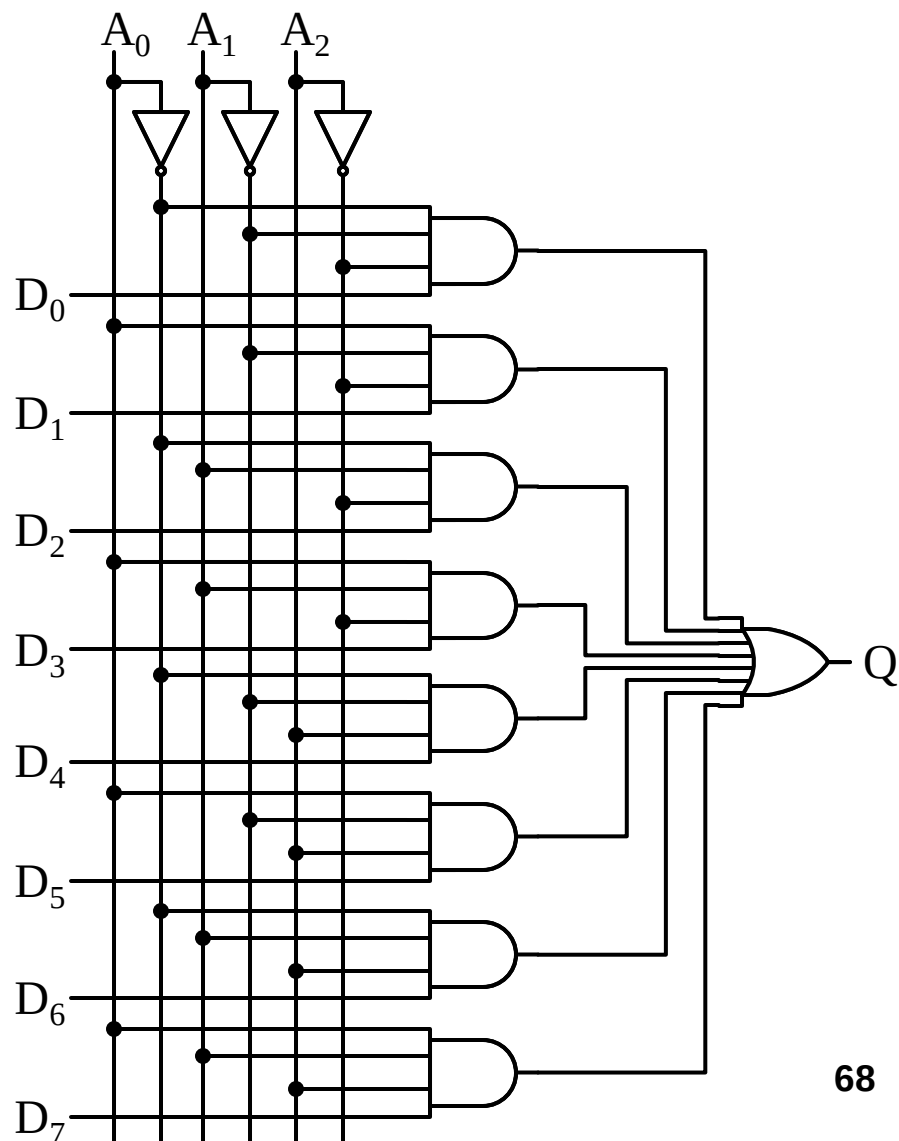
功能表

A_2	A_1	A_0	Q
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

$$Q = \bar{A}_2 \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_2 \bar{A}_1 A_0 D_1 + \bar{A}_2 A_1 \bar{A}_0 D_2 + \bar{A}_2 A_1 A_0 D_3 \\ + A_2 \bar{A}_1 \bar{A}_0 D_4 + A_2 \bar{A}_1 A_0 D_5 + A_2 A_1 \bar{A}_0 D_6 + A_2 A_1 A_0 D_7$$

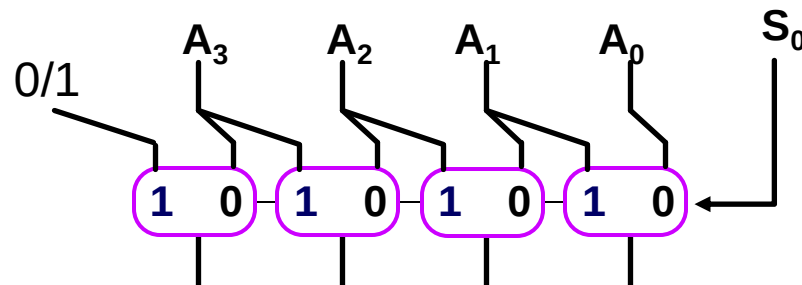
两级与或逻辑电路

$$\begin{aligned} Q = & \bar{A}_2 \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_2 \bar{A}_1 A_0 D_1 \\ & + \bar{A}_2 A_1 \bar{A}_0 D_2 + \bar{A}_2 A_1 A_0 D_3 \\ & + A_2 \bar{A}_1 \bar{A}_0 D_4 + A_2 \bar{A}_1 A_0 D_5 \\ & + A_2 A_1 \bar{A}_0 D_6 + A_2 A_1 A_0 D_7 \end{aligned}$$

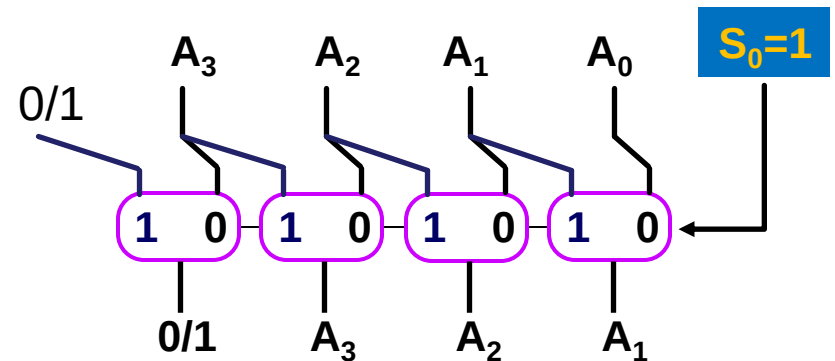
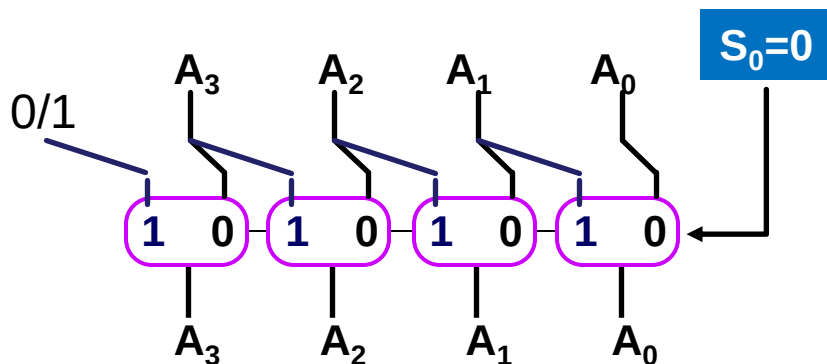


多路选择器应用：移位器

- 采用2:1多路选择器构造的右移位器



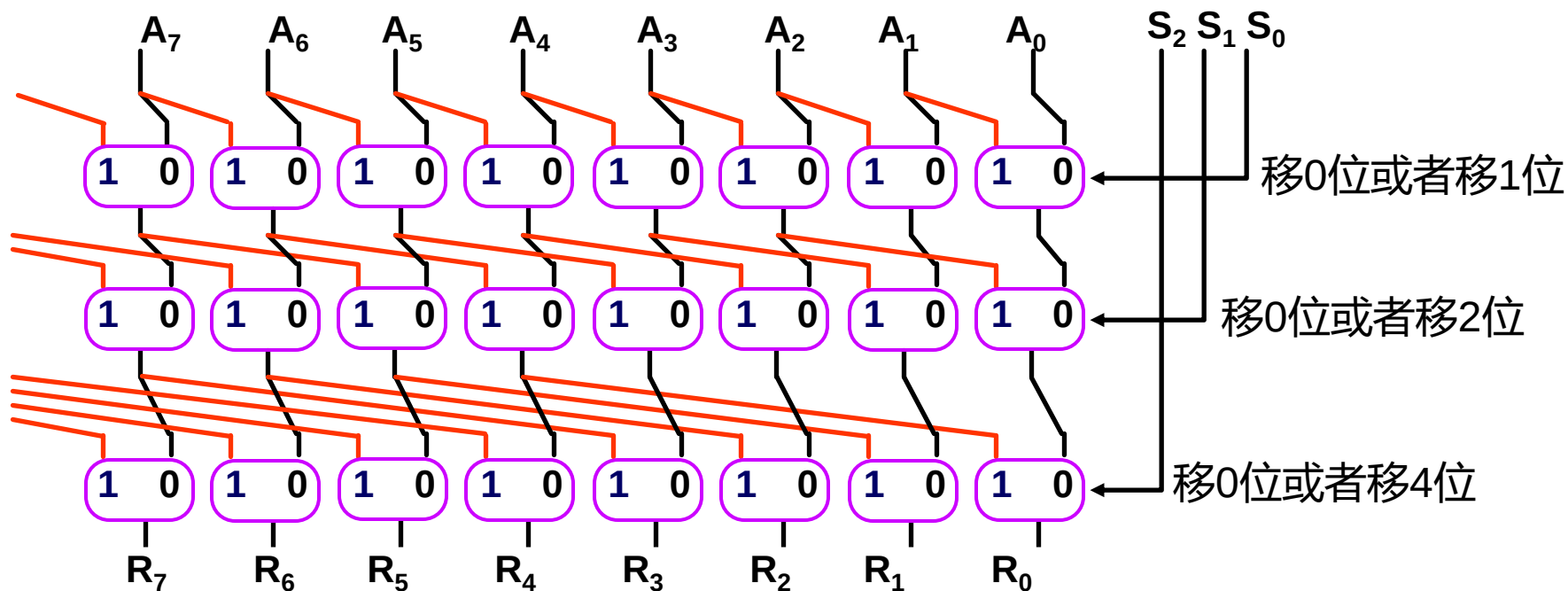
Note: 每一个紫色的框为一个1-bit的多路选择器；
当 $S_0=0$ 时，下方的输出选择0号输入端的数据；
当 $S_0=1$ 时，下方的输出选择1号输入端的数据；



多路选择器应用：移位器

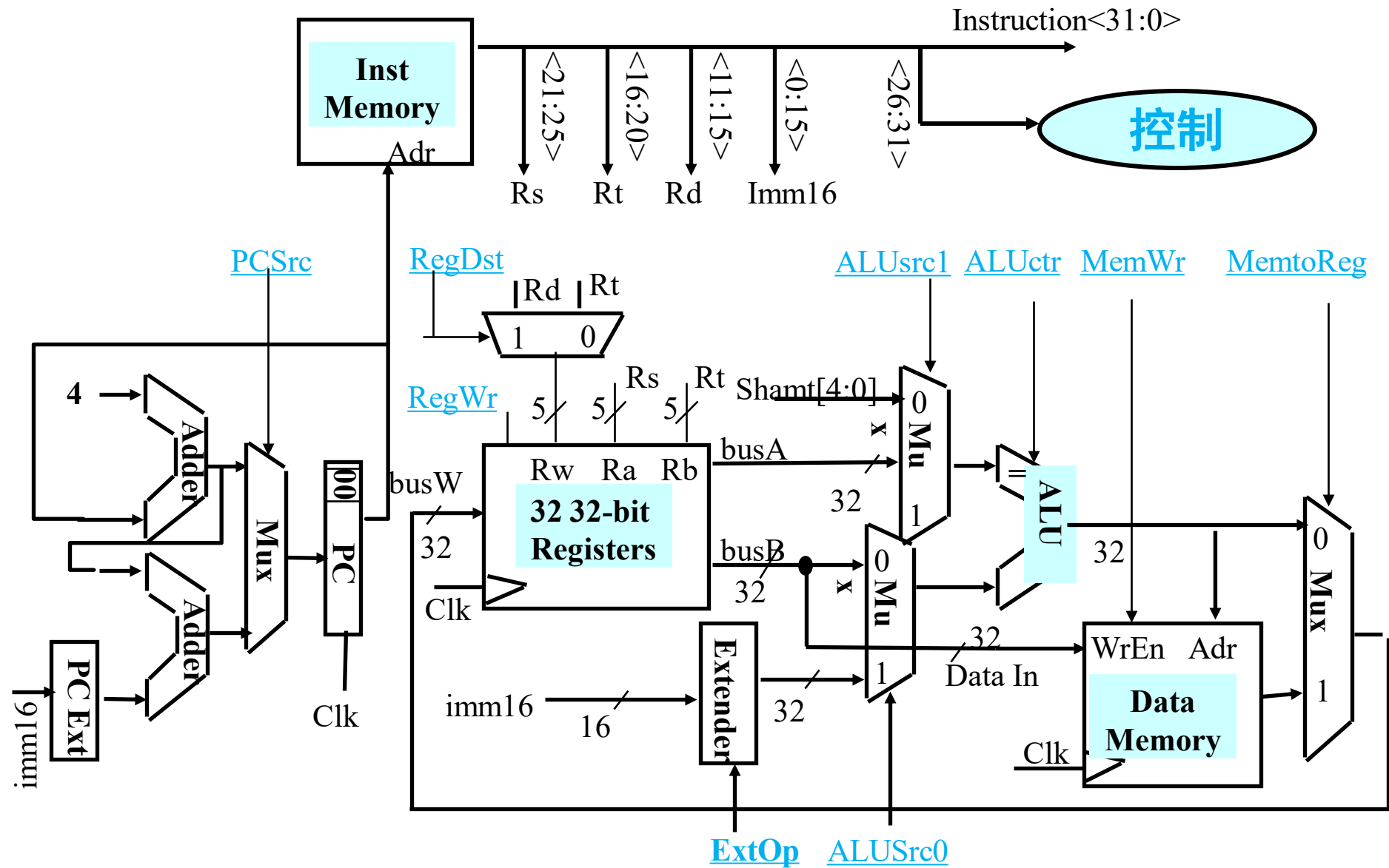
- 采用2:1多路选择器构造的8位右移位器

– Input: $A_7A_6\dots A_0$, $S_2S_1S_0$; Output: $R_7R_6\dots R_0$



思考：最高位MSB的输入？支持0-31位移位器需要多少级？

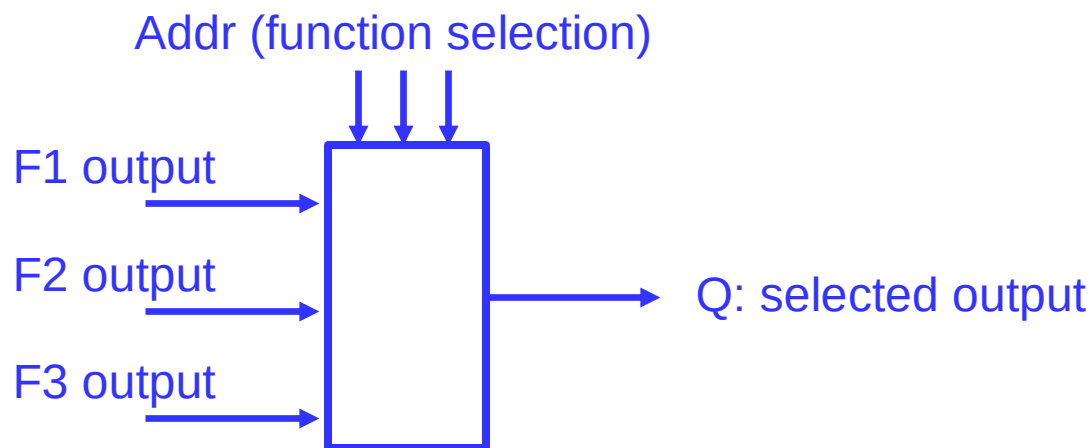
CPU中选择器的应用



选择器应用：逻辑函数

- 多路选择器 vs 查找表 (LUT)
- n个地址端的多路选择器 → n变量的逻辑函数
 - 将函数输入变量作为地址端
 - 多路选择器的数据输入端按函数输出值预先赋值

$$Q = \bar{A}_2 \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_2 \bar{A}_1 A_0 D_1 + \bar{A}_2 A_1 \bar{A}_0 D_2 + \bar{A}_2 A_1 A_0 D_3 \\ + A_2 \bar{A}_1 \bar{A}_0 D_4 + A_2 \bar{A}_1 A_0 D_5 + A_2 A_1 \bar{A}_0 D_6 + A_2 A_1 A_0 D_7$$



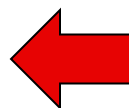
用8选1 MUX实现一个3变量逻辑函数

$$F=f(A,B,C)$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$D_0=D_1=D_4=D_7=0$$

$$D_2=D_3=D_5=D_6=1$$



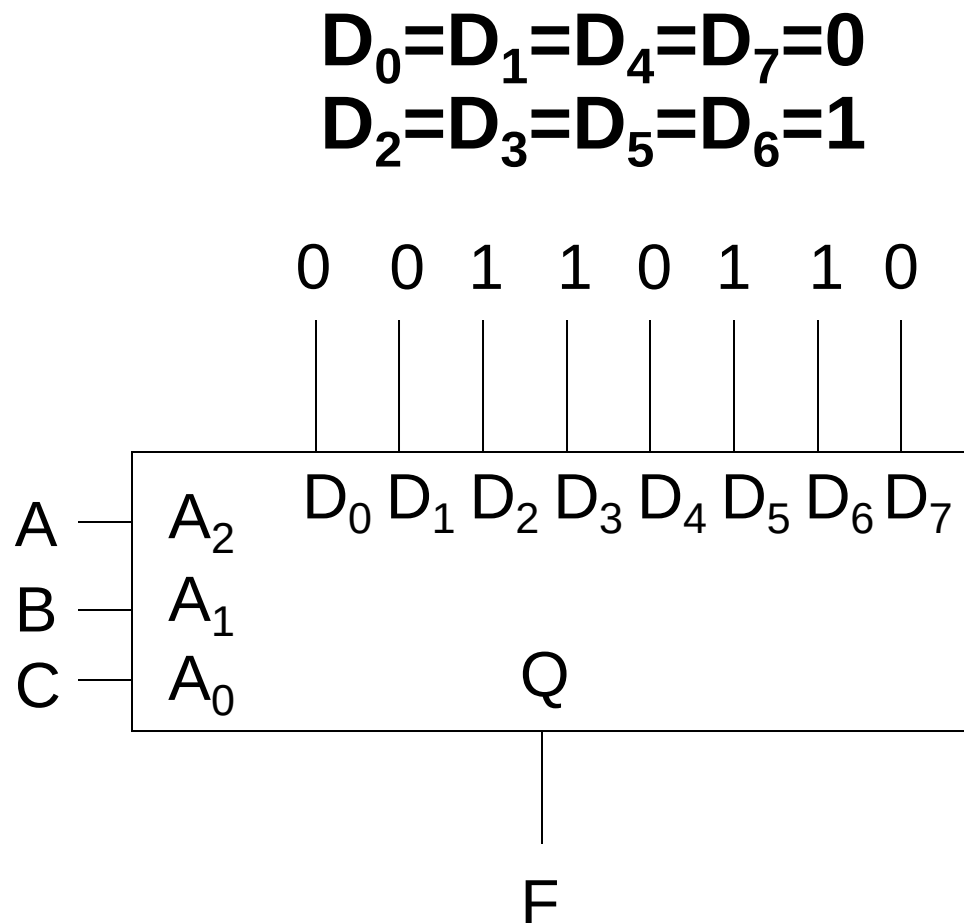
A ₂	A ₁	A ₀	Q
0	0	0	D ₀
0	0	1	D ₁
0	1	0	D ₂
0	1	1	D ₃
1	0	0	D ₄
1	0	1	D ₅
1	1	0	D ₆
1	1	1	D ₇

$$Q = \bar{A}_2 \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_2 \bar{A}_1 A_0 D_1 + \bar{A}_2 A_1 \bar{A}_0 D_2 + \bar{A}_2 A_1 A_0 D_3 \\ + A_2 \bar{A}_1 \bar{A}_0 D_4 + A_2 \bar{A}_1 A_0 D_5 + A_2 A_1 \bar{A}_0 D_6 + A_2 A_1 A_0 D_7$$

用8选1 MUX实现一个3变量逻辑函数

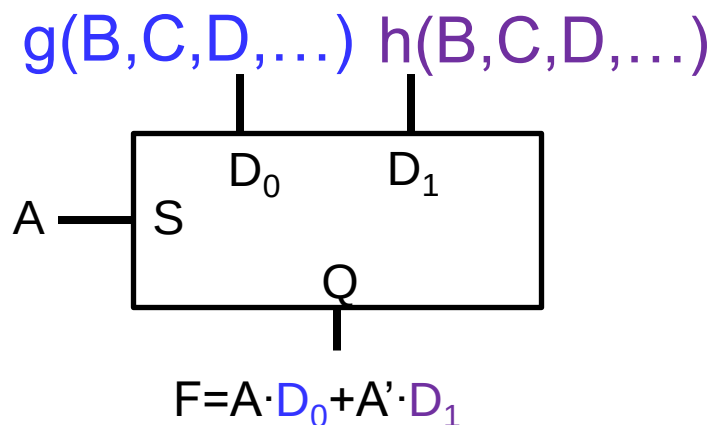
$$Q=f(A,B,C)$$

A_2	A_1	A_0	Q
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7



用n比特的多路选择器表示m比特的函数

- 将函数的n个输入变量作为MUX的地址端
- 其余的输入变量对应的子函数存入MUX的选通列表
- 1比特多路选择器例子:



$$\begin{aligned} F &= f(A, B, C, D, \dots) \\ &= A \cdot g(B, C, D, \dots) + A' \cdot h(B, C, D, \dots) \end{aligned}$$

用8选1 MUX实现一个4变量逻辑函数

$$F=f(A,B,C,D)=ABC+BC'D'+AB'C'+AB'D+B'C'D$$

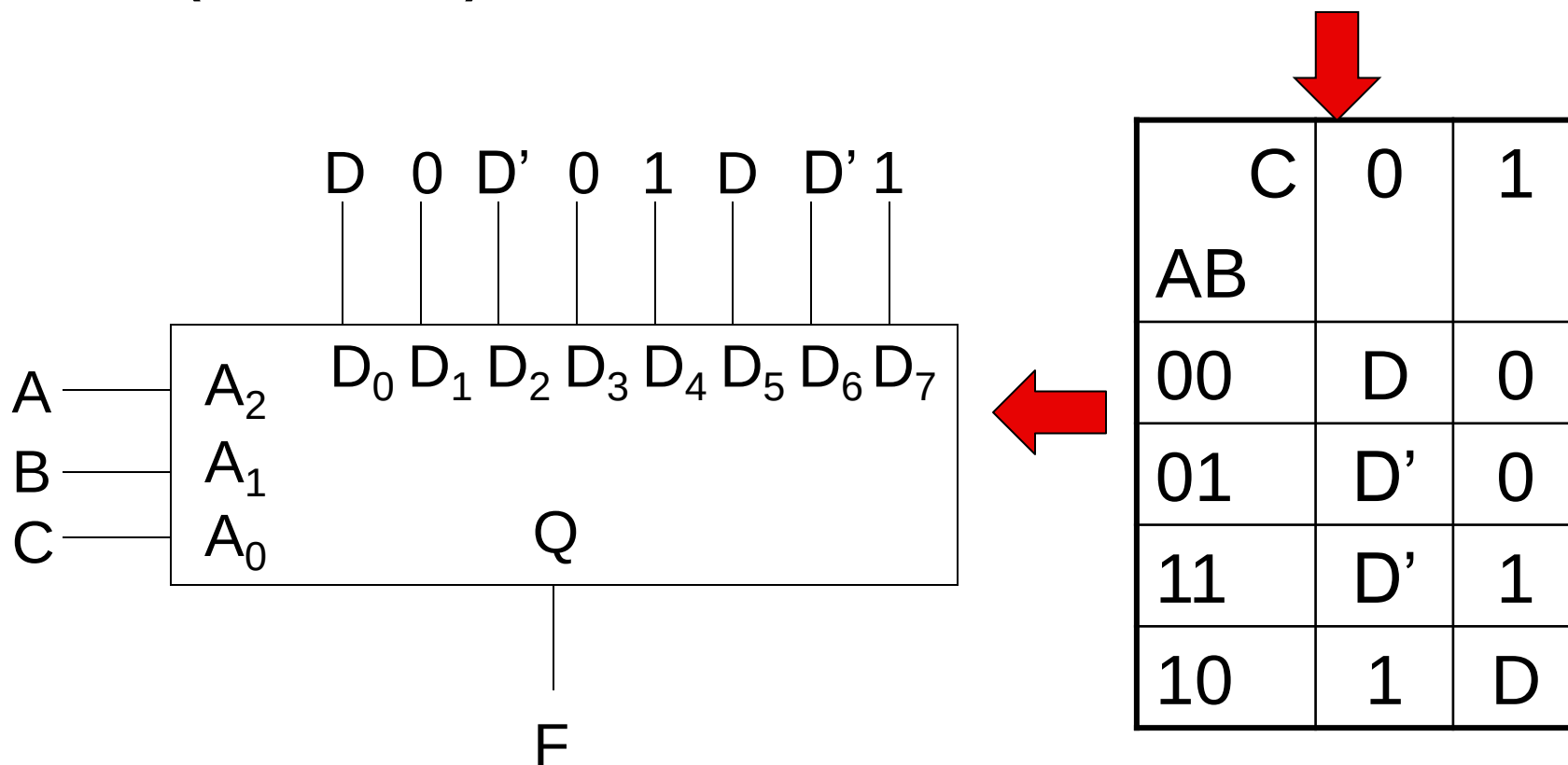
CD	00	01	11	10
AB				
00	0	1	0	0
01	1	0	0	0
11	1	0	1	1
10	1	1	1	0



C	0	1
AB		
00	D	0
01	D'	0
11	D'	1
10	1	D

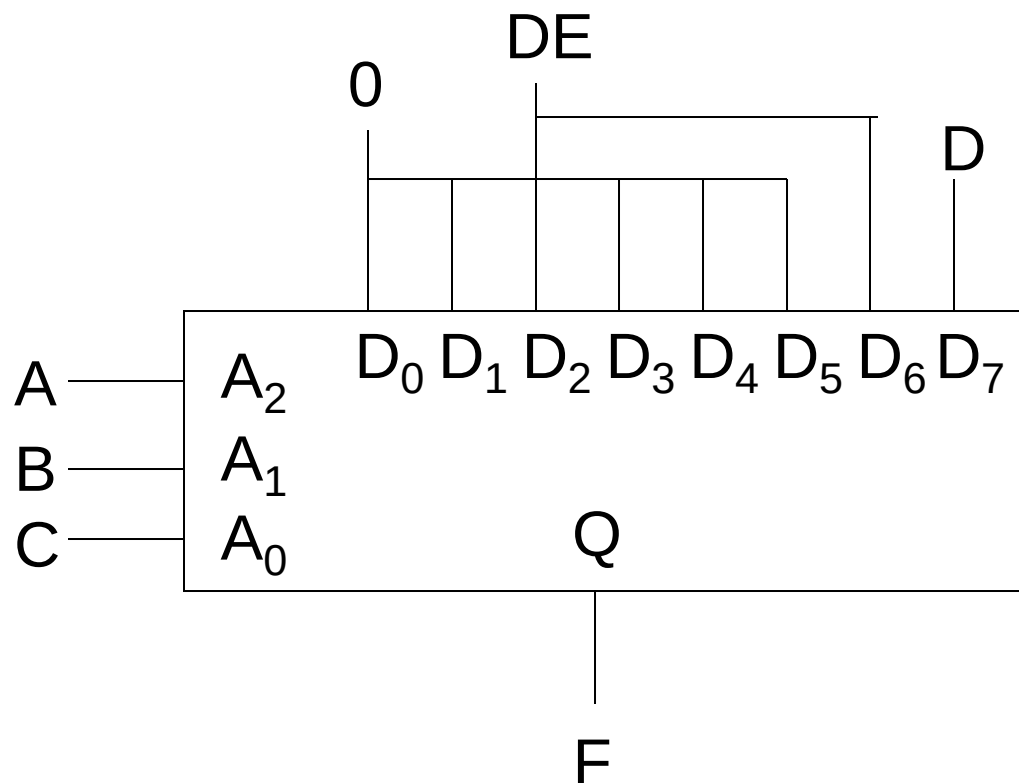
用8选1 MUX实现一个4变量逻辑函数

$$F=f(A,B,C,D)=ABC+BC'D'+AB'C'+AB'D+B'C'D$$



用8选1 MUX实现5变量逻辑函数

$$F=f(A,B,C,D,E)=ABCD+BC'DE$$



C	0	1
AB		
00	0	0
01	DE	0
11	DE	D
10	0	0

常用组合逻辑

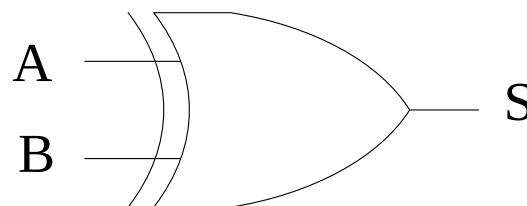
- 编码器/译码器
- 多路选择器
- 加法器adder
 - 在计算机中加、减、乘、除均可转换为加法运算
 - 加法器分类
 - 半加器：无进位信号（只有2个本位数相加）
 - 全加器：有进位输入输出信号、2个本位数

半加器

- Half Adder
- 逻辑抽象
 - ✓ 输入2个: A, B
 - ✓ 输出1个: S
- 列出真值表

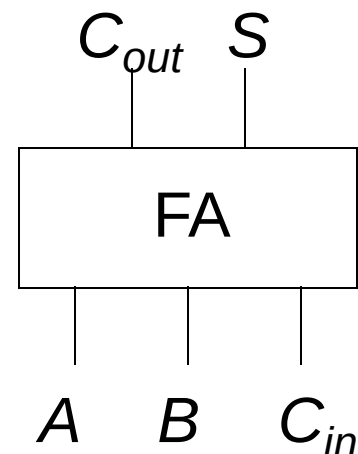
A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

$$S = A'B + AB' = A \oplus B$$



设计1比特全加器

Input			Output	
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



- 也称为(3,2) adder
- 逻辑函数
 - $C_{out} = A \cdot C_{in} + B \cdot C_{in} + A \cdot B$
 - $S = A \oplus B \oplus C_{in}$

多位加法器

$$A: 1010 + B: 1011 = \mathbf{1}0101$$

$A_3A_2A_1A_0$

1 0 1 0

$A_0: 0$

$A_1: 1$

$A_2: 0$

$A_3: 1$

$B_0: 1$

$B_1: 1$

$B_2: 0$

$B_3: 1$

$C_0: 0$

$C_1: 0$

$C_2: 1$

$C_3: 0$

$B_3B_2B_1B_0$

1 0 1 1

$S_0: 1$

$S_1: 0$

$S_2: 1$

$S_3: 0$

$C_1: 0$

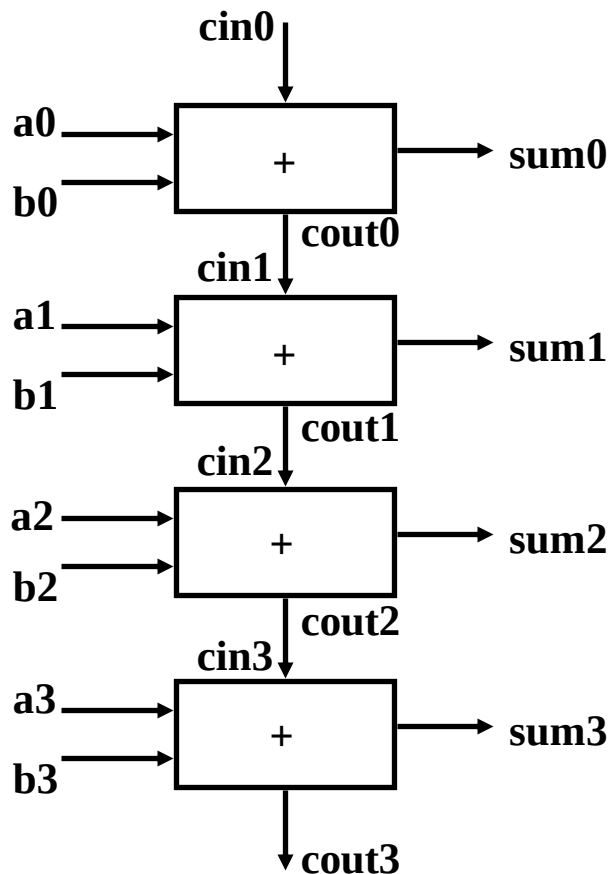
$C_2: 1$

$C_3: 0$

$C_4: 1$

串行加法器

- 也称“行波进位加法器” (Ripple Carry Adder)



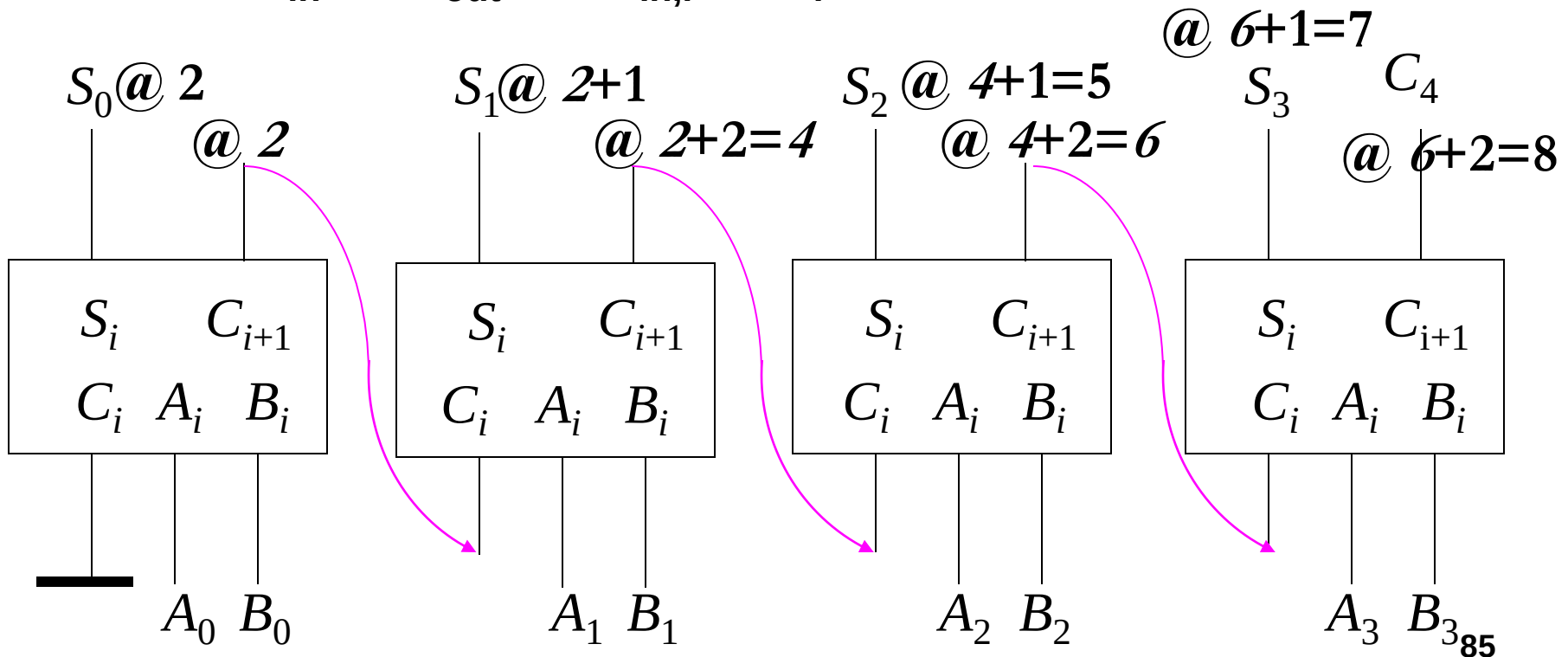
- 优点: 简单, 紧凑
- 缺点: 慢

串行加法器: 延时

• 延时假设

- 2: A_i to C_{out} ; 2: A_i to S_i
- 2: C_{in} to C_{out} ; 1: $C_{in,i}$ to S_i

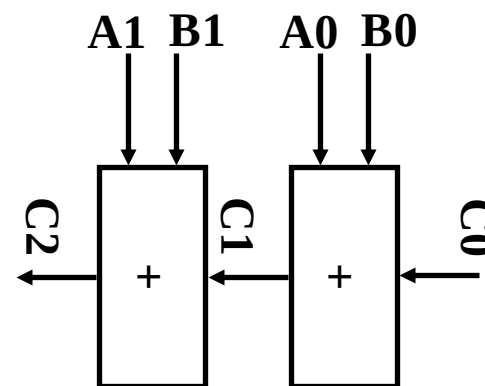
@2表示在时刻2信号才到达



可以更快些吗？

- 两比特加法器的进位：

- $C_{i+1} = A_i C_i + B_i C_i + A_i B_i$
- $C_2 = A_1 C_1 + B_1 C_1 + A_1 B_1$
- $C_1 = A_0 C_0 + B_0 C_0 + A_0 B_0$



- 将 C_1 代入 C_2 ：

- $C_2 = A_1 \cdot A_0 \cdot B_0 + A_1 \cdot A_0 \cdot C_0 + A_1 \cdot B_1 \cdot C_0 + B_1 \cdot A_0 \cdot B_0 + B_1 \cdot A_0 \cdot C_0 + B_1 \cdot A_0 \cdot C_0 + B_1 \cdot A_1$
- C_2 不需要等待 C_1 计算结束后才能计算，可直接算
- C_2 表达式虽更复杂，但可能比 C_1 、 C_2 先后算更快

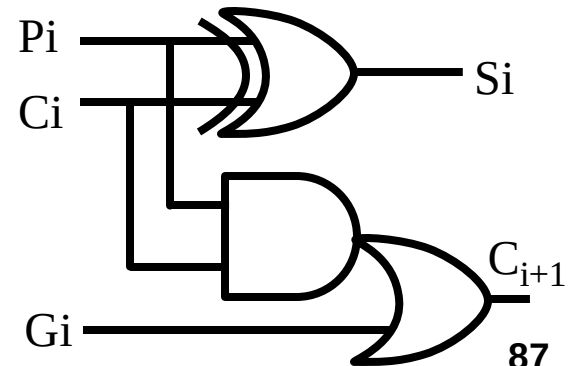
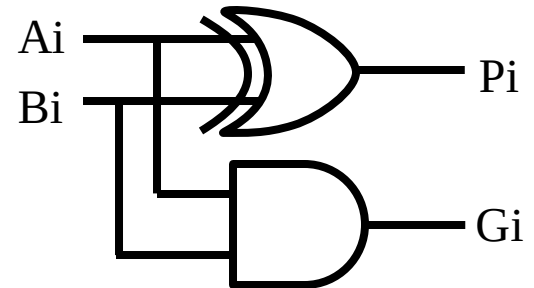
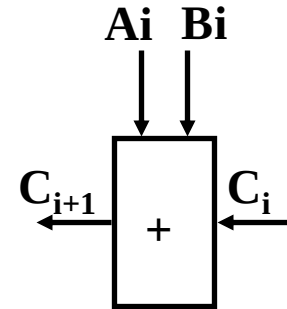
- 给 C_3 、 C_4 也递推下去吗？

对于 $C_3, C_4, \dots$?

已知: $C_{i+1} = B_i C_i + A_i B_i + A_i C_i$

定义: $P_i = A_i \oplus B_i$ $G_i = A_i B_i$

则: $C_{i+1} = P_i C_i + G_i$
 $S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$



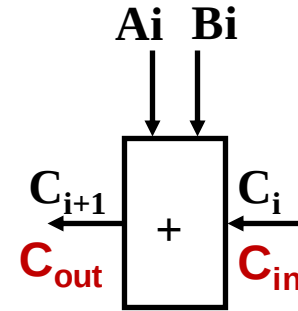
如何理解Pi 和 Gi?

$$P_i = A_i \oplus B_i$$

Carry-bit
Propagation
进位传播

$$G_i = A_i B_i$$

Carry-bit
Generation
进位生成



$$\textcircled{1} A + B \Rightarrow C_{out} = 1$$

$$G_i = A_i B_i = 1$$

$$\textcircled{1} A + B + C_{in} \Rightarrow C_{out} = 1$$

$$P_i C_{in} = 1$$

问题: 什么时候 C_{out} 为 1?

两种情形:

- ✓ $A=B=1 \rightarrow C_{out}=1$, 无论 C_{in} 取值
- ✓ $\{AB = '10' \text{ or } '01'\}$ and $C_{in}=1$
- $A=B=0$ 不会使得 $C_{out}=1$

$$C_{i+1} = P_i C_i + G_i$$

Carry lookahead adder (CLA,超前进位)

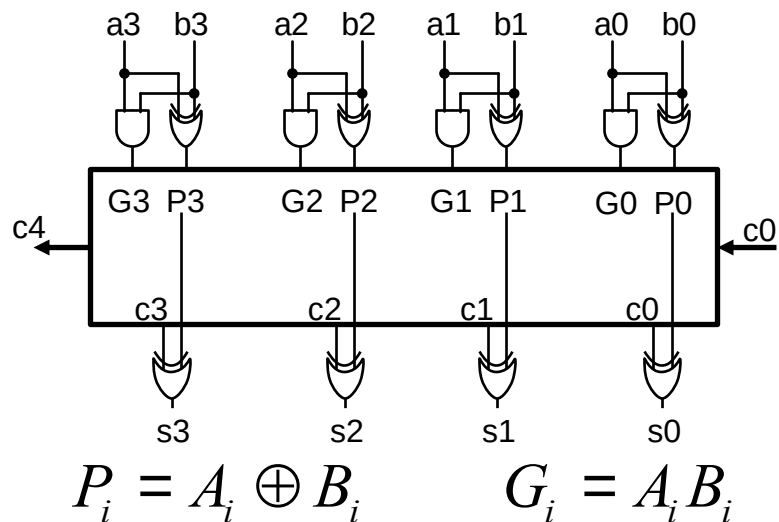
利用 P_i 、 G_i 递推得到 C_{out} : $C_2 = G_1 + P_1 C_1$

$$C_1 = G_0 + P_0 C_0, \quad C_2 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

4比特 CLA 加法器



You can calculate C_1 - C_4 directly now!

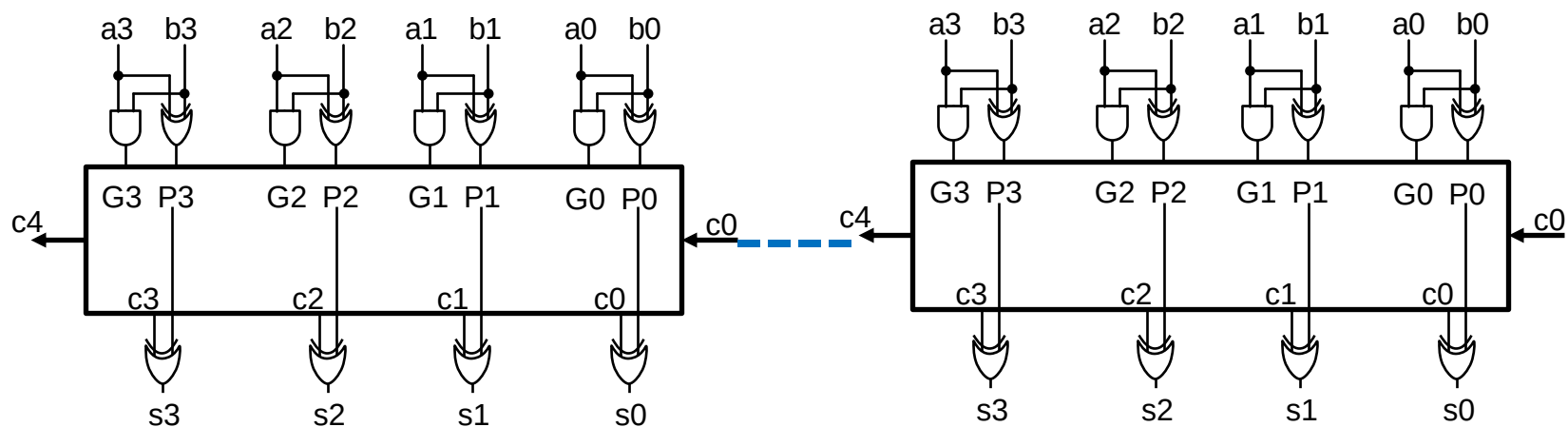
- P_i/G_i 1级门延时 (from A/B)
- C_1 - C_4 由 $P_i/G_i/C_0$ 产生, 3级门延时 from A/B, 2 from C_0
- S_0 , 2级门延时 from A/B, 1 from C_0 ; $S_1/S_2/S_3$, 4级门延时 from A/B, 3 from C_0 $S_i = P_i \oplus C_i$

对更多比特数进行超前进位？

- P&G 变得更加复杂!
- CLA 的比特数通常小于等于4比特
- 问题: 如果需要64比特的加法器呢?

两个4位超前进位加法器级联

- P_i/G_i 1级门延时 (from A/B)
 - C_1-C_4 由 $P_i/G_i/C_0$ 产生, 3级门延时 from A/B, 2 from C_0
 - S_0 , 2级门延时 from A/B, 1 from c_0
 - $S_1/S_2/S_3$, 4级门延时 from A/B, 3 from c_0
- $C4_1st: 2$
 $C4_2nd: 2+2=4$
 $S7: C4_1st+3=5$

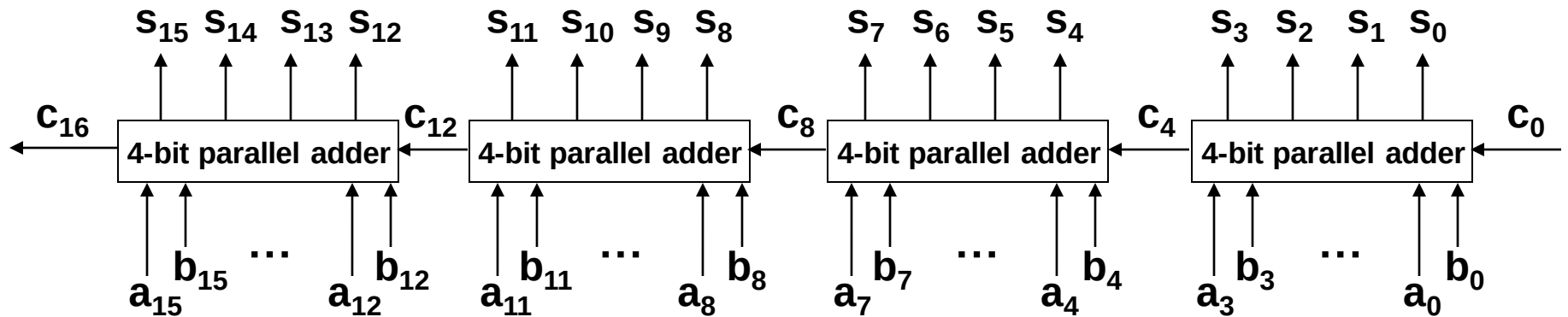


问: A/B不变, S_7 会在第一级 C_0 输入改变后, 经过几级门延时输出? (5)

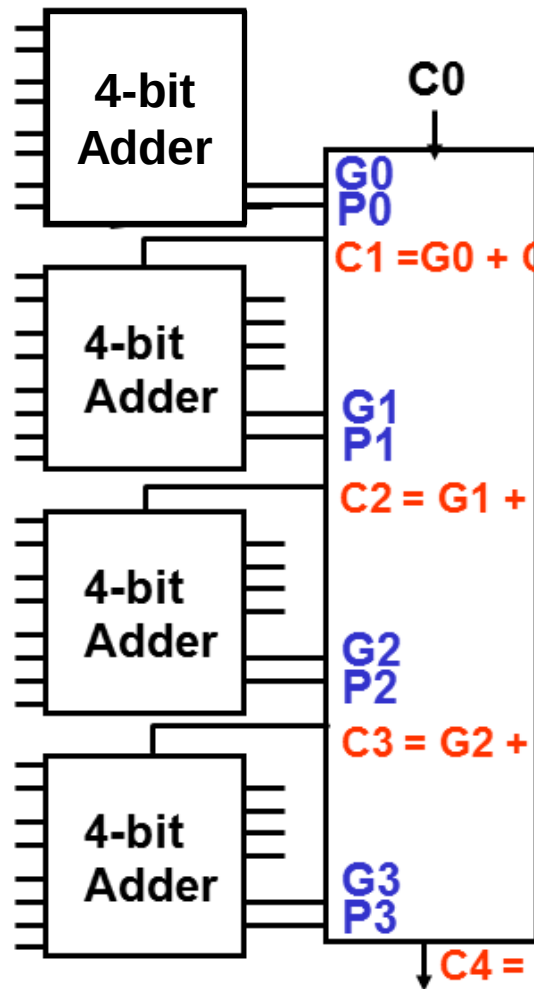
第二级的 c_4 呢? (4)

更多4比特CLA级联

4比特加法器内部：并行
4比特加法器之间：串行



组内并行、组间并行进位加法器



$$C_1 = G_0 + C_0 \cdot P_0$$

$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

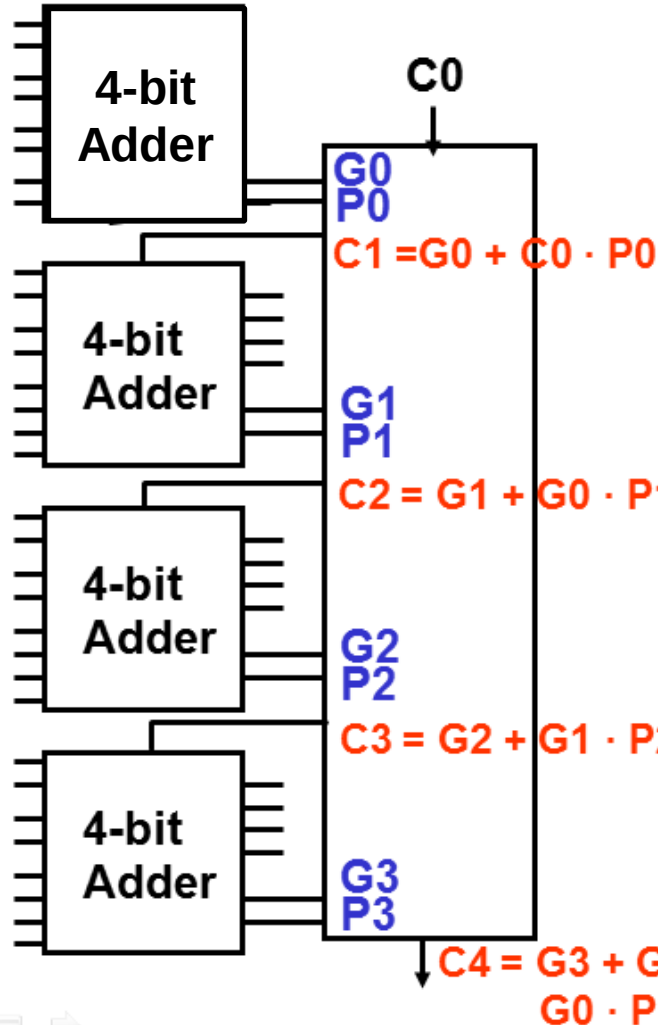
$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

关键：
将 P_i 和 G_i 的概念从1比特
扩展到1组(4比特)

$G_0=1$: the 1st 4-bit adder gets $C_{out,1}=1$, i.e. $C_1=1$
 $P_0=1$: the 1st 4-bit adder propagates C_{in} to C_{out}

组内并行、组间并行进位加法器

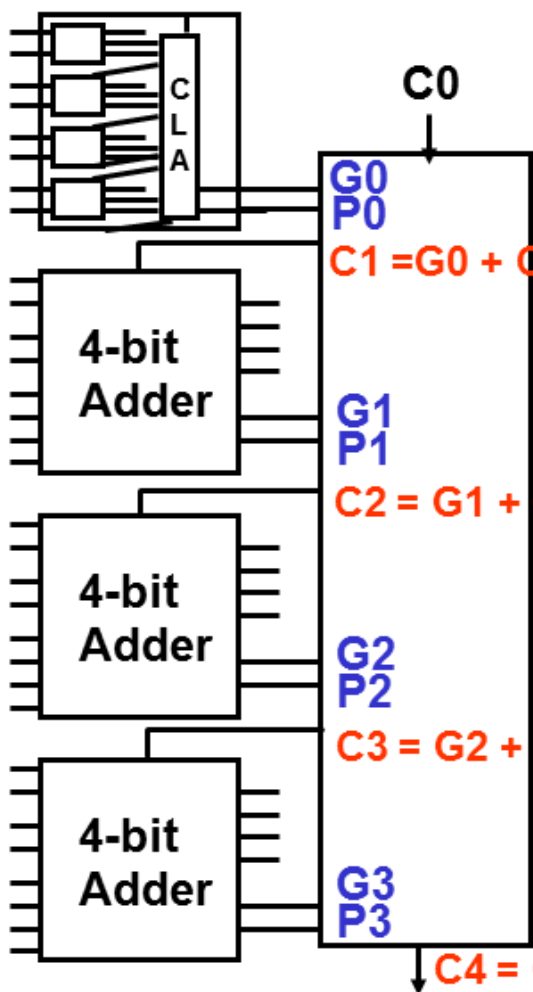


核心:

- 在一个4-bit CLA内部, 对每一位加法
 $C_{out}=1 \leftrightarrow G=1 \text{ or } P \cdot C_{in}=1$
- 每一个4-bit 加法器会生成 $C_{out}=1$, 如果:
 - ✓ 本身4位数相加得到 $C_{out}=1$ (对应于 $G=1$)
 - ✓ 或传递来自一个低4位加法器的进位 C_{in} (对应于 $P=1$)

$G_0=1$: the 1st 4-bit adder gets $C_{out,1}=1$, i.e. $C_1=1$
 $P_0=1$: the 1st 4-bit adder propagates C_{in} to C_{out}

组内并行、组间并行进位加法器



1个4比特加法器组的Pi和Gi:

$$G_0 = g_3 + g_2 \cdot p_3 + g_1 \cdot p_3 \cdot p_2 + g_0 \cdot p_3 \cdot p_2 \cdot p_1$$

$$P_0 = p_3 \cdot p_2 \cdot p_1 \cdot p_0$$

$$G_1 = g_7 + g_6 \cdot p_7 + g_5 \cdot p_7 \cdot p_6 + g_4 \cdot p_7 \cdot p_6 \cdot p_5$$

$$P_1 = p_7 \cdot p_6 \cdot p_5 \cdot p_4$$

$$G_2 = g_{11} + g_{10} \cdot p_{11} + g_9 \cdot p_{11} \cdot p_{10} + g_8 \cdot p_{11} \cdot p_{10} \cdot p_9$$

$$P_2 = p_{11} \cdot p_{10} \cdot p_9 \cdot p_8$$

$$G_3 = g_{15} + g_{14} \cdot p_{15} + g_{13} \cdot p_{15} \cdot p_{14} + g_{12} \cdot p_{15} \cdot p_{14} \cdot p_{13}$$

$$P_3 = p_{15} \cdot p_{14} \cdot p_{13} \cdot p_{12}$$

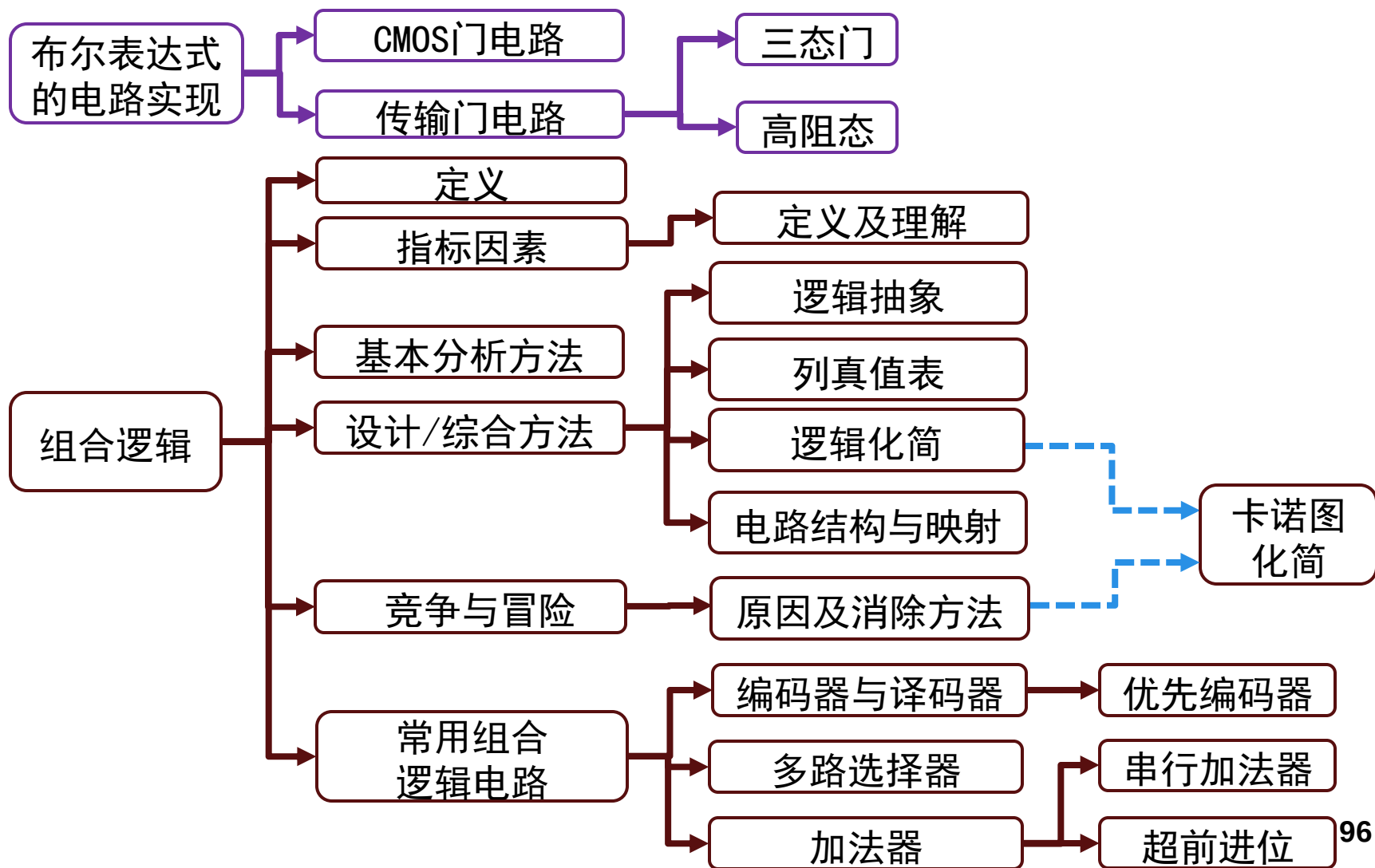
$$C_1 = G_0 + C_0 \cdot P_0$$

$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

组合逻辑总结



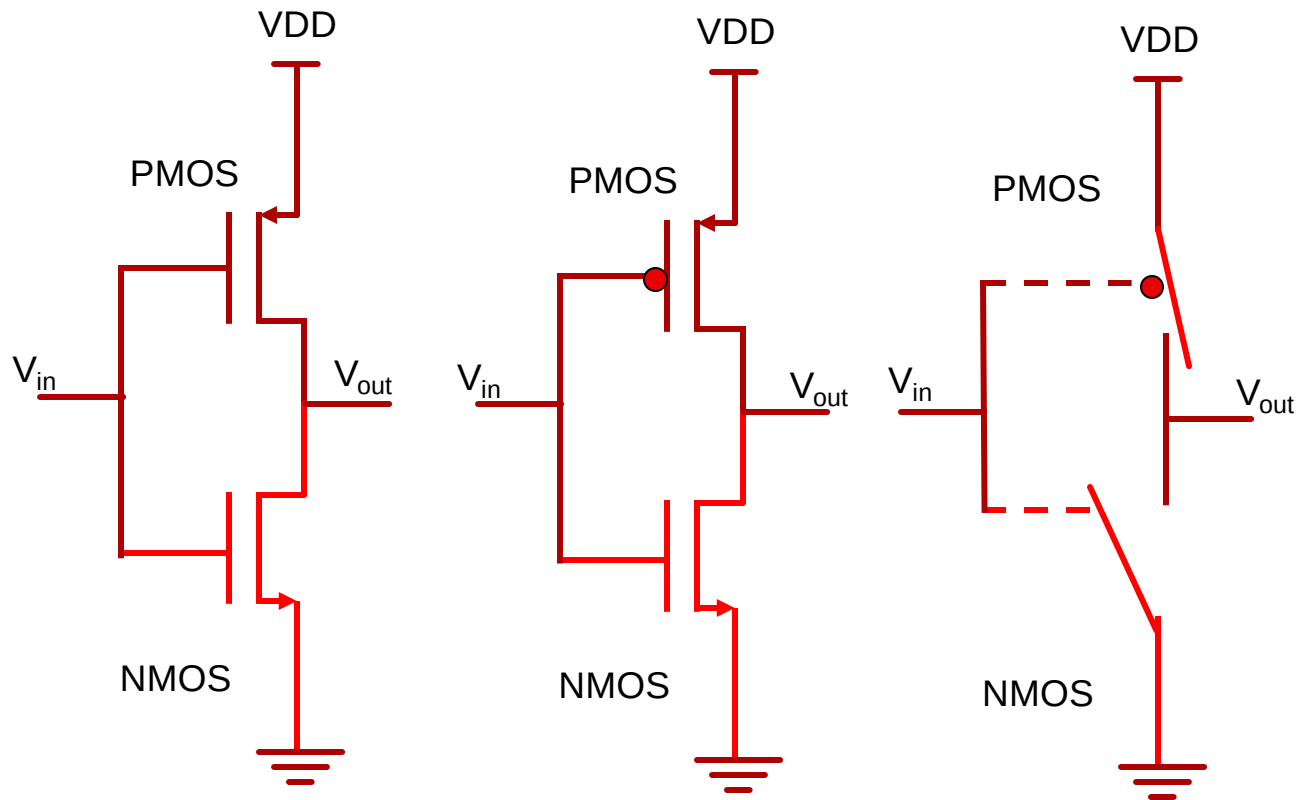
作业

- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区

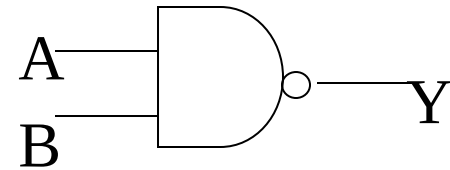
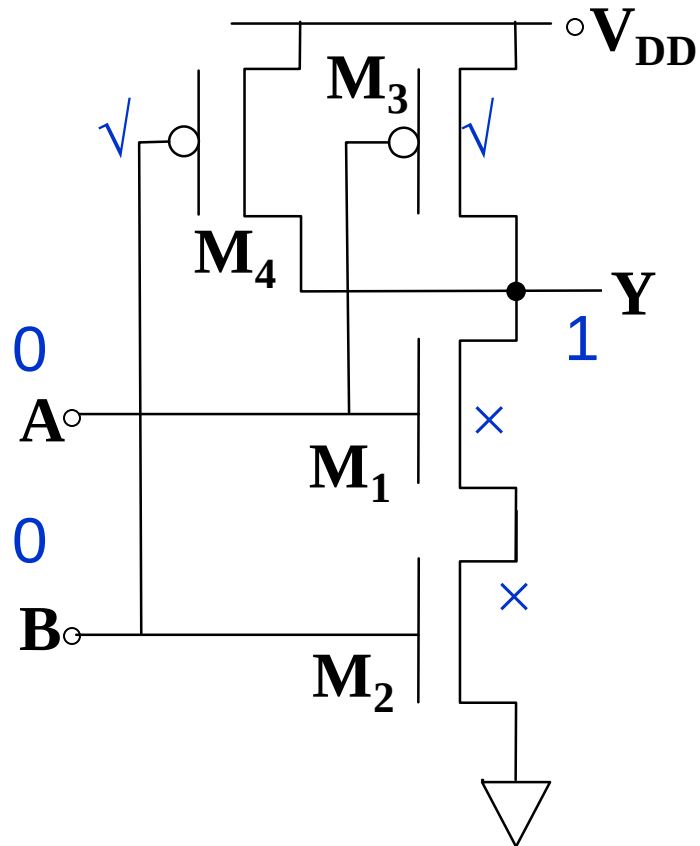
补充阅读内容

- CMOS逻辑门电路结构

CMOS 非门

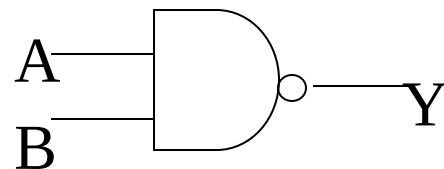
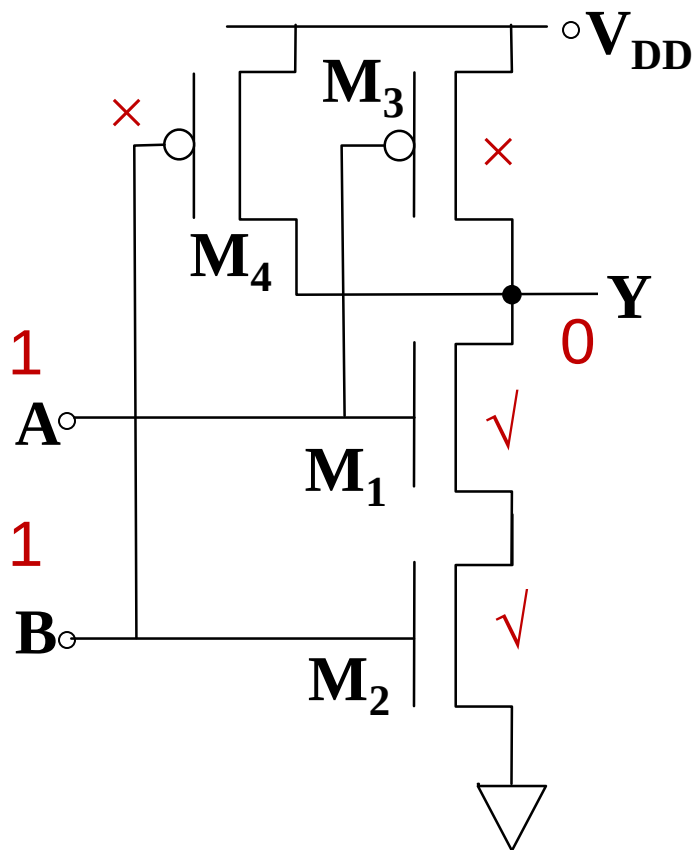


CMOS与非门NAND



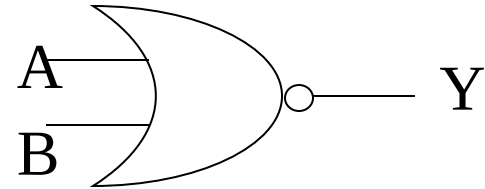
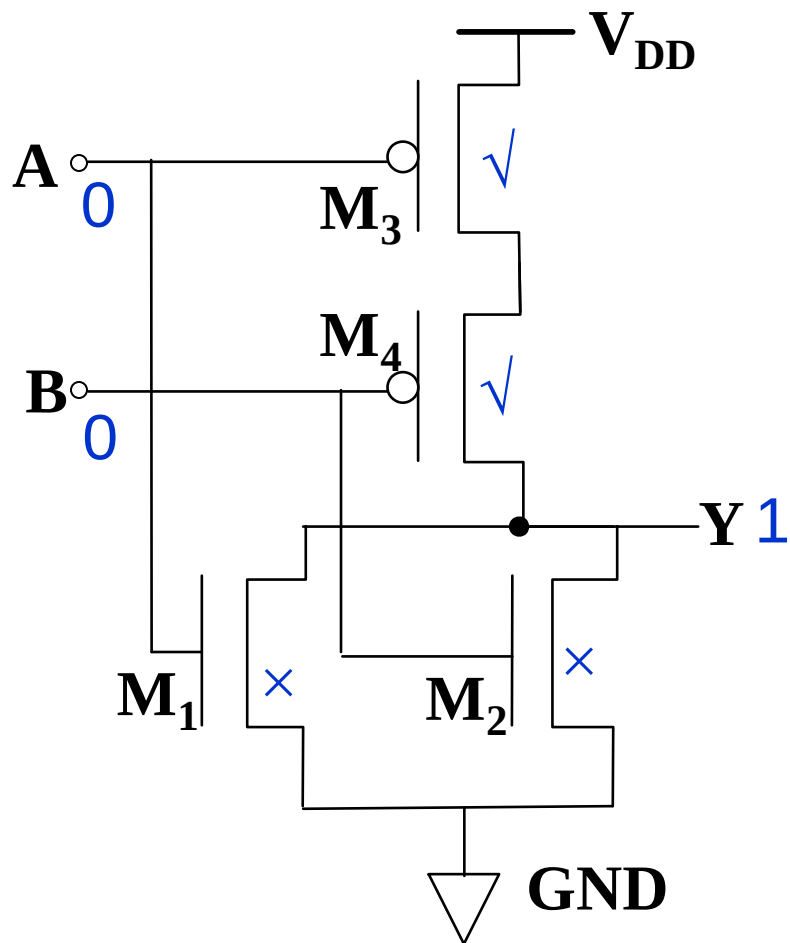
A	B	M_1	M_2	M_3	M_4	Y
L	L	Off	Off	On	On	H

CMOS与非门NAND



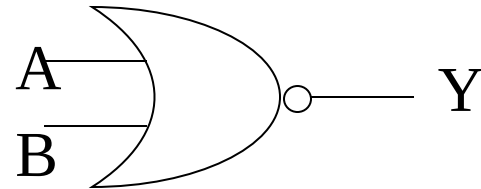
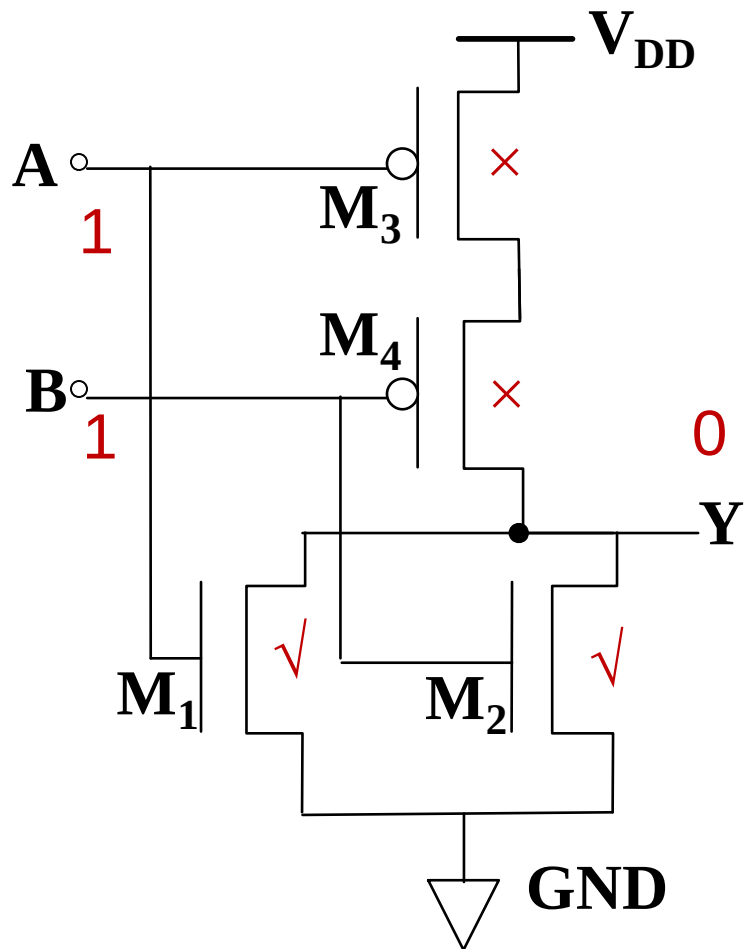
A	B	M_1	M_2	M_3	M_4	Y
L	L	Off	Off	On	On	H
L	H	Off	On	On	Off	H
H	L	On	Off	Off	On	H
H	H	On	On	Off	Off	L

CMOS或非门NOR



A	B	M_1	M_2	M_3	M_4	Y
L	L	Off	Off	On	On	H

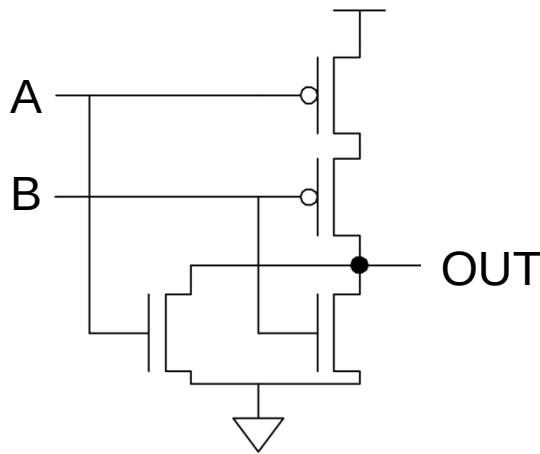
CMOS或非门NOR



A	B	M ₁	M ₂	M ₃	M ₄	Y
L	L	Off	Off	On	On	H
L	H	Off	On	On	Off	L
H	L	On	Off	Off	On	L
H	H	On	On	Off	Off	L

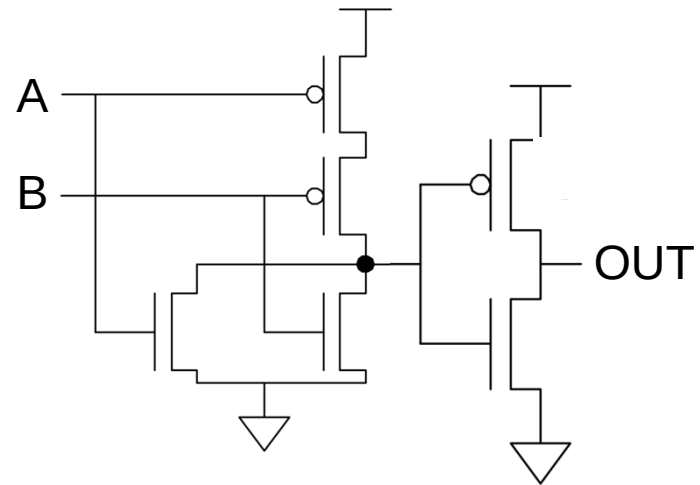
CMOS与门/或门

- NAND/NOR + 非门
- 注意输出处的反相器



$$\text{OUT} = A \text{ NOR } B$$

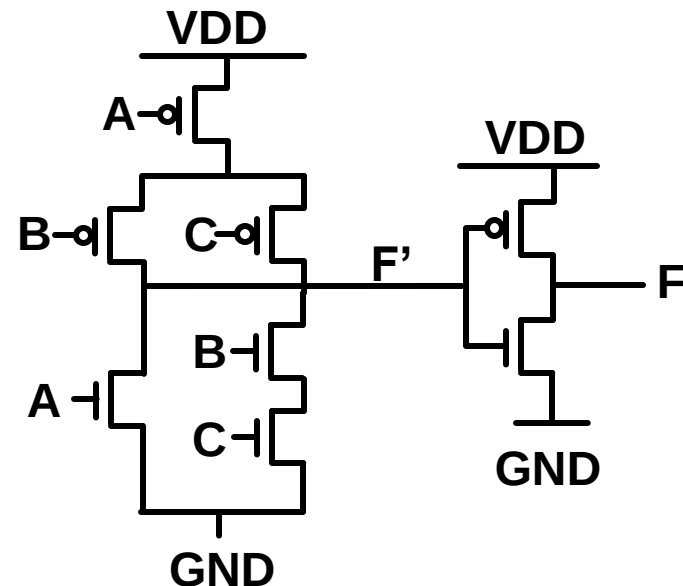
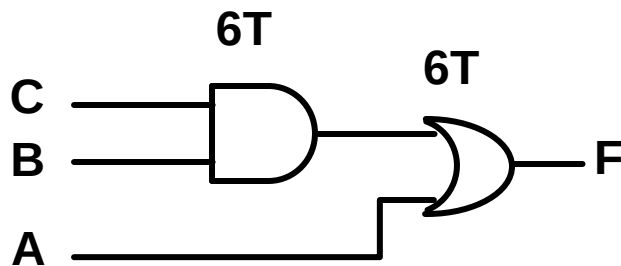
$$\text{逻辑功能: } f(x_1, x_2, \dots) = \overline{g(x_1, x_2, \dots)}$$



$$\text{OUT} = A \text{ OR } B$$

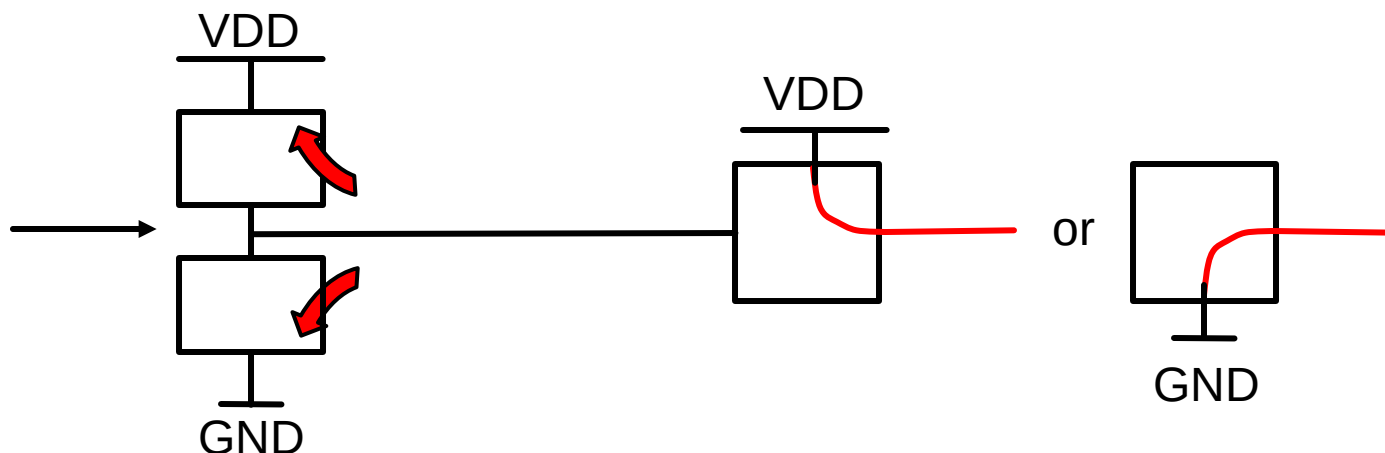
定制你的CMOS逻辑门

- 功能: $f(x_1, x_2, \dots) = \overline{g(x_1, x_2, \dots)}$
- $f(x_1, x_2, \dots) = 1$, PMOS on, NMOS off
- $f(x_1, x_2, \dots) = 0$, NMOS on, PMOS off
- 例子: $f(A, B, C) = A + BC$



CMOS逻辑门

- 开放式讨论



输入信号控制栅极，没有静态输入电流。

输出不会悬空！

输出连接VDD（通过pmos）或者GND（通过NMOS）。